



Trimble Financials Payroll (Lista)

System Development Plan Review

Architect: Jeff Doolittle



Vision

Construction Payroll made simple to help business owners mitigate risks, fulfill compliance obligations, and effectively manage project profitability.



Mission

Leverage an AI augmented Software Development Lifecycle (SDLC) to create a collection of reusable software building blocks that engineers can efficiently assemble into microservices to support Construction Payroll APIs, features and applications.



Nature of the Business

To provide a critical risk management function that ensures compliance with legal and contractual requirements for payroll-related payments and reporting, while accurately assessing and allocating job costs for construction projects.



Glossary

- **SMB**: Small to Medium Sized Business
- **Certified**: Work subject to Davis-Bacon Act (Prevailing Wage)
- **Gross to Net**: The process of determining net employee pay after removing calculated taxes and deductions
- **Burden**: The full labor-related job cost



Objectives

- Target SMB + First
- AI Augmentation
- Leverage Expertise and Experience
- Early Path to Revenue (SaaS Startup approach)
- Infrastructure Investment



Observations

- In-house domain expertise
- Ongoing Alignment
- Lean Team
- SaaS Startup Approach
- Accounting Integration



Operational Concepts

- Modern architecture approach
 - System Design
 - Detailed Design
 - Specification Driven Development (SDD)
- Employ rigorous Quality Assurance and Quality Control
 - Automation, Repeatability and Reversibility
- Use existing work log collection capabilities (Traqspera)
- Consider outsourcing calculations to a 3rd party
- Embrace modern approaches to system integration



Areas of Change (Business)

- **Compliance**
 - Regulatory
 - Contractual
 - Parameters
 - Calculations
 - Classifications
- **Configuration**
 - Rates
 - Benefits
- **Costs**
 - Configuration
 - Tracking
 - Allocation
 - Sync
- **Analysis**
 - Detection
 - Optimization
- **Reporting**
 - Auditing
 - Formats



Areas of Change (Technical)

- **Messaging**
- **Streaming**
- **Monitoring**
- **Notifications**
- **Administration**
- **Integrations**
 - Internal
 - 3rd Party
- **Security**
 - Authentication
 - Authorization



Core Use Cases

- **Configure**
 - Onboard
 - Schedule
 - Rate
- **Collect**
 - Track
 - Correct
 - Approve
 - Send
- **Process**
 - Reconcile
 - Prepare
 - Submit
 - Calculate
 - Report
 - Pay/Transfer



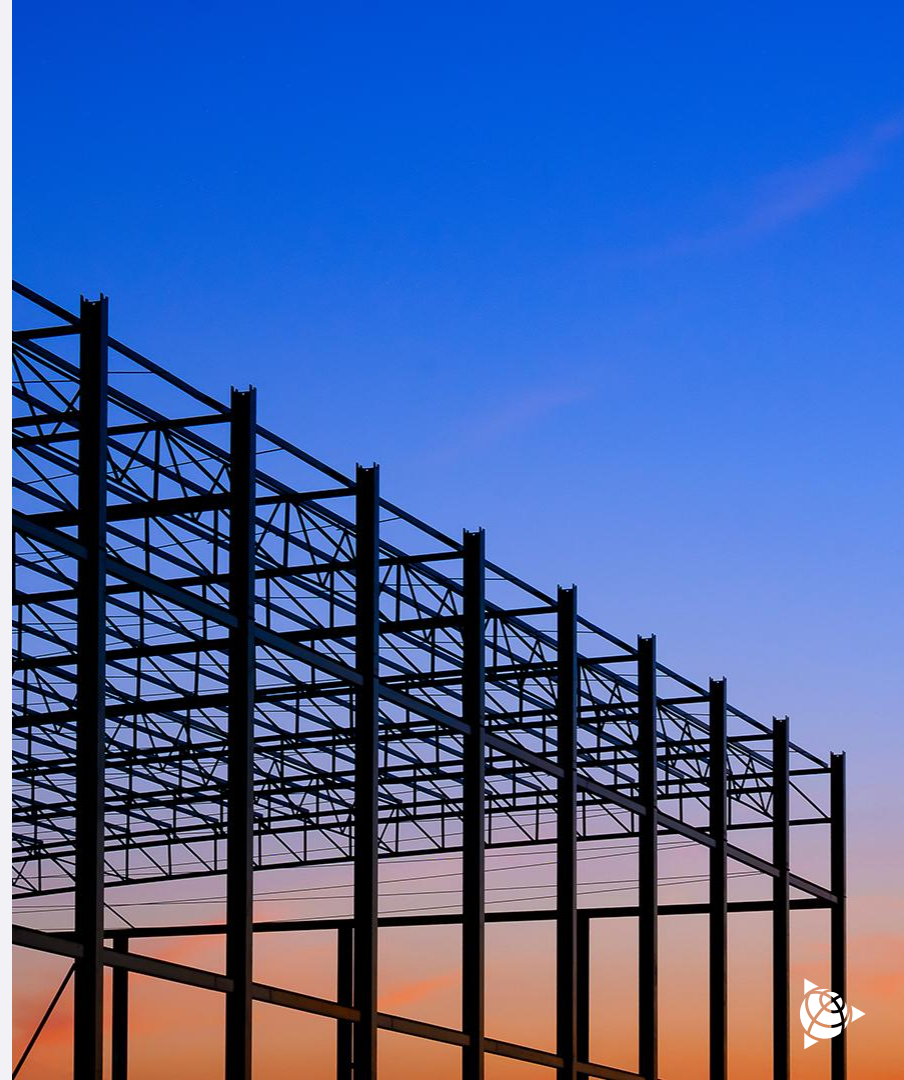
Supporting Use Cases

- **Integrate**
 - Ingest/Digest
 - Stage/Emit
- **Analyze**
 - Detect
 - Optimize
 - Benchmark
- **Notify**
 - Warn
 - Alert

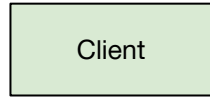


Architecture

Components



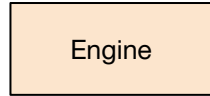
Component Taxonomy



- Originator of a request
- An app, API or another system



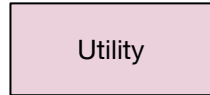
- Orchestrator
- Encapsulates use case orchestration
- Single point of entry for microservice



- Logic
- Encapsulates actions
- Rules, algorithms, calculations, aggregation, etc.



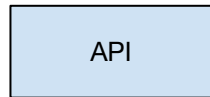
- Access
- Encapsulates access to external resources
- Produces business-related, not schema related DTOs
- Shields system from resource access mechanism



- Supportive components/frameworks/infrastructure
- No business logic



- Physical system resource
- Database, database schema

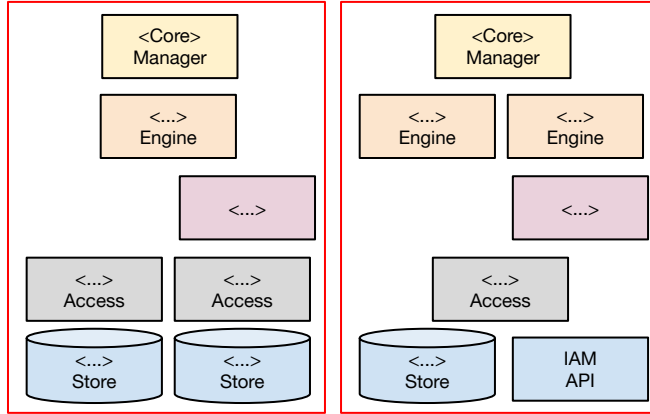


- External system, 3rd party SDK, blob storage



Mapping Subsystems to DDD Sub-domains

Core



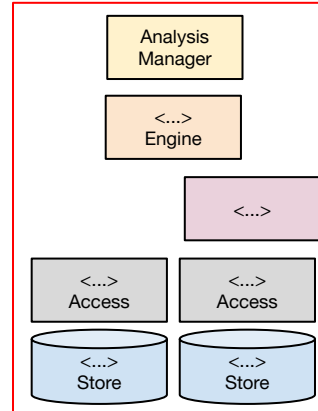
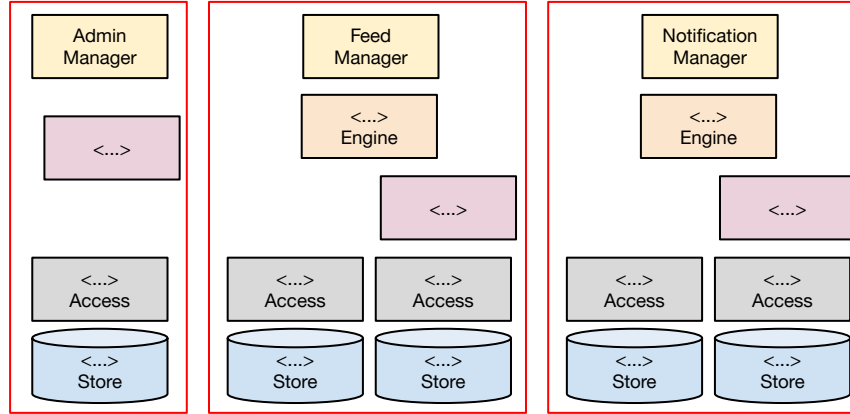
Most systems have **one or two Core use cases**. A system with 3 is a very large system. One with 4 or 5 is a monster system. I have never seen or heard of an example with 6 or more Core use cases.

Each Core use case maps to one subsystem represented by each Manager that performs use cases related to the respective Core use cases, including the components necessary for the Manager to provide the ability to execute each use case.

Many business systems have requirements provided by **common Supporting use cases**. There are **four classic subsystems** that arrive from common volatilities related to these Supporting use cases.

Admin encapsulates the volatility in configuration management.
Feed encapsulates the volatility of integration with other systems.
Notification encapsulates the volatility of asynchronous and/or out-of-band communications to humans.
Analysis encapsulates the volatility in complex calculations that provide business insights.

Supporting

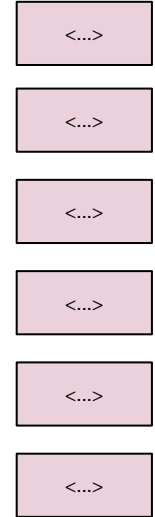


You should expect to own every aspect of the implementation of your **Core** subsystems.

You should expect varying levels of implementation effort in the **Supporting** subsystems, which you should seek to minimize.

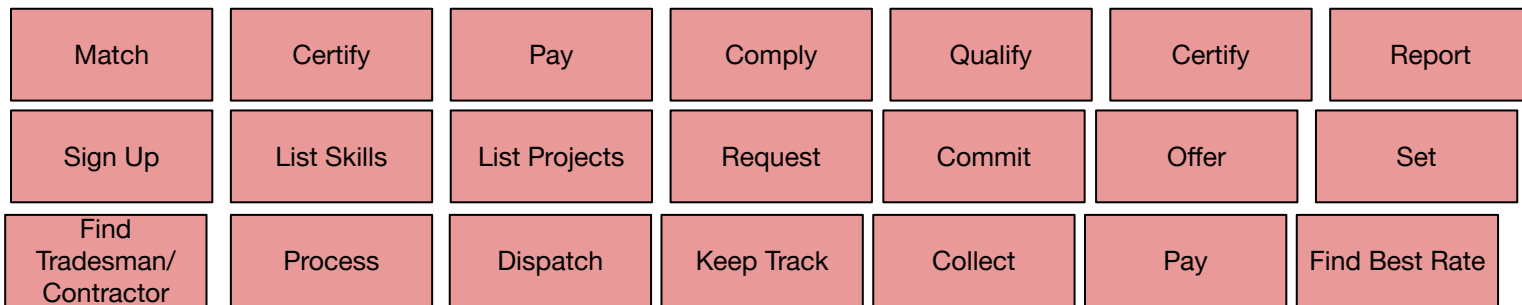
You should expect the barest minimum of implementation effort in the **Generic** subsystems.

Generic

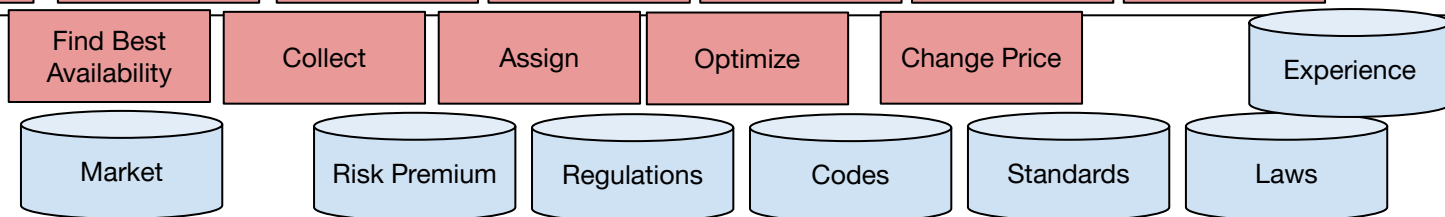


The Anti Design of TradeMe

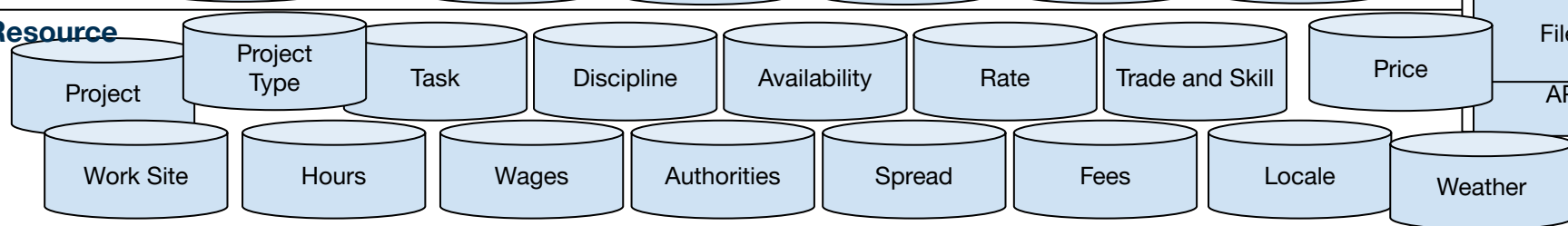
Logic



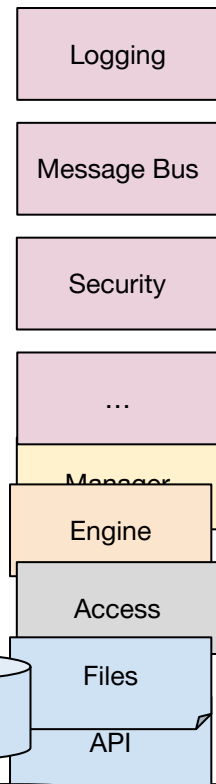
Access



Resource



Utilities



Client

Collect Web Client

Collect Mobile Client

Payroll Client

Employee Client

ERP

Time Collector

Timer

Cloud System

Utilities

Telemetry

Security

Messaging

Streaming

Encryption

Comms

Inference

Logic

Admin Manager

Collection Manager

Process Manager

Feed Manager

Notification Manager

Analysis Manager

Validation Engine

Compliance Engine

Transform Engine

Format Engine

Detection Engine

“Lookup” Engine

Filter Engine

Suggestion Engine

Optimization Engine

Access

Work Log Access

Settings Access

Rule Access

Accounting Access

Report Access

Profile Access

Batch Access

Reference Access

Bank Access

Resource

Work Log Storage

Settings Storage

Rule Storage

Accounting API

Report Storage

Profile Storage

Batch Storage

Reference Cache

Bank APIs

Documents



Client

Collect Web Client

Collect Mobile Client

Payroll Client

Employee Client

ERP

Time Collector

Timer

Cloud System

Utilities

Telemetry

Security

Messaging

Streaming

Encryption

Comms

Inference

Logic

Admin Manager

Collection Manager

Process Manager

Feed Manager

Notification Manager

Validation Engine

Compliance Engine

Transform Engine

Format Engine

Filter Engine

Access

Work Log Access

Settings Access

Rule Access

Report Access

Accounting Access

Profile Access

Batch Access

Reference Access

Bank Access

Resource

Work Log Storage

Settings Storage

Rule Storage

Report Storage

Accounting API

Profile Storage

Batch Storage

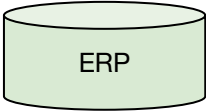
Reference Cache

Documents

Bank APIs

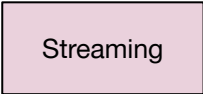


Client

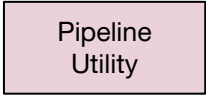


Pipelines

Utilities



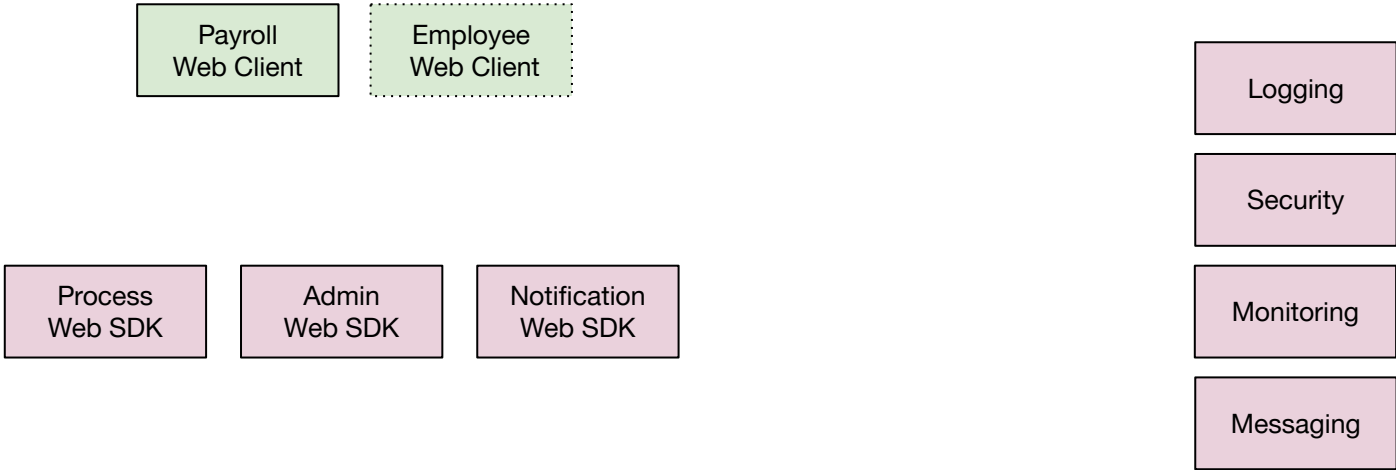
Logic

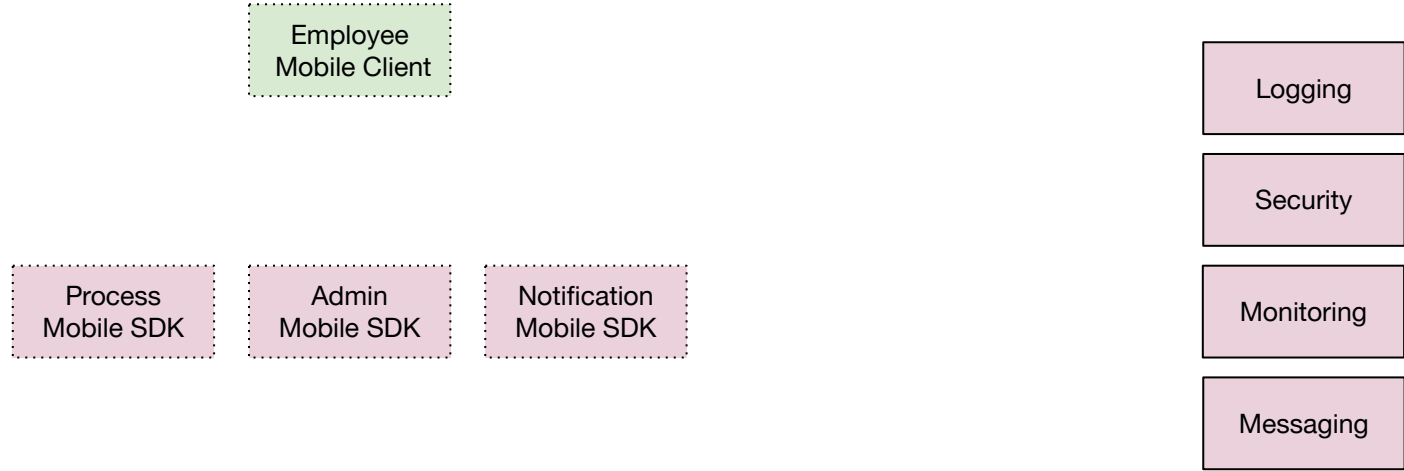


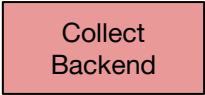
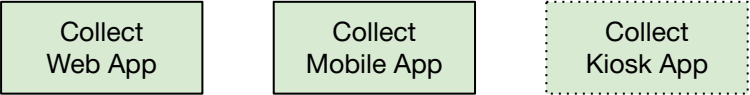
Access

Resource



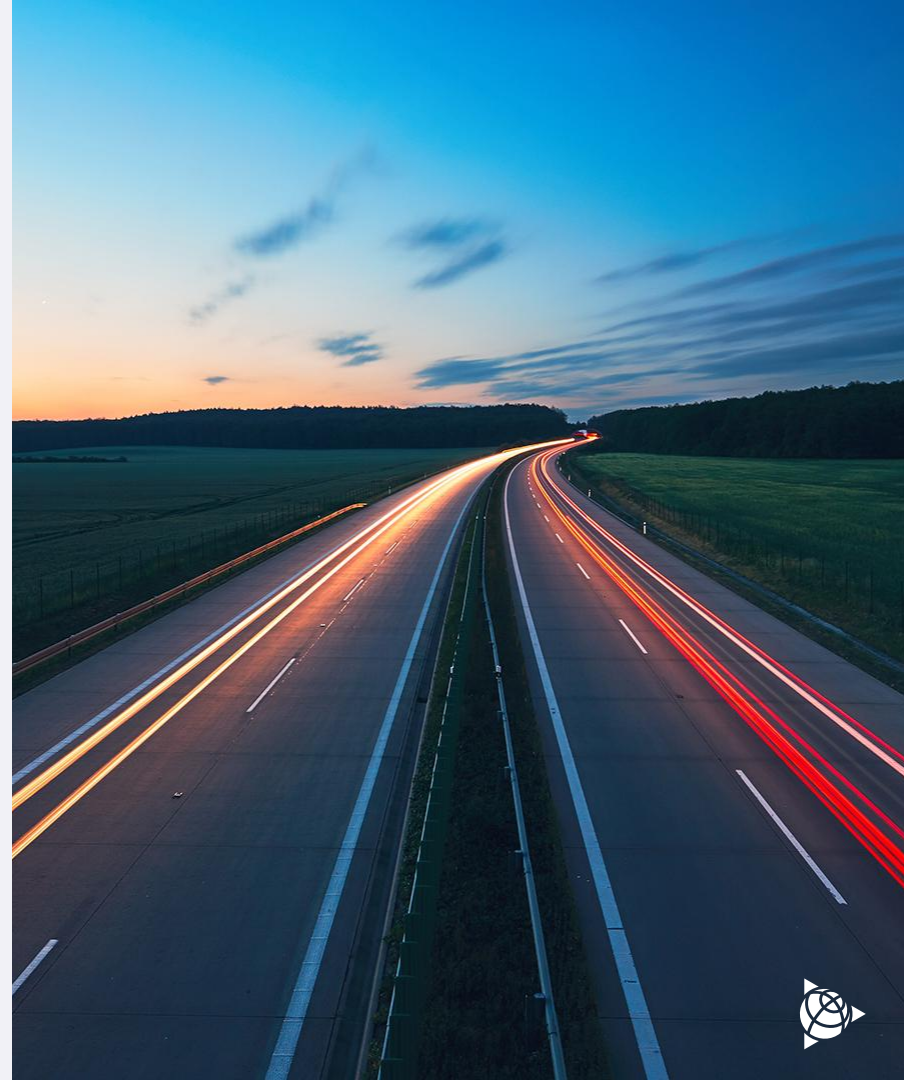




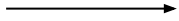


Architecture

Runtime Interactions



Key



Synchronous (request/response, send/receive)



Asynchronous (transient, pub/sub)



Contextual Boundary (relative to diagram)

- Security boundary
- Process boundary
- Machine boundary
- Environment boundary



Patterns



Send/Receive

Pattern



One manager can queue a command to another manager. When it does so, it implies that the Message Bus is functioning as the transport mechanism.

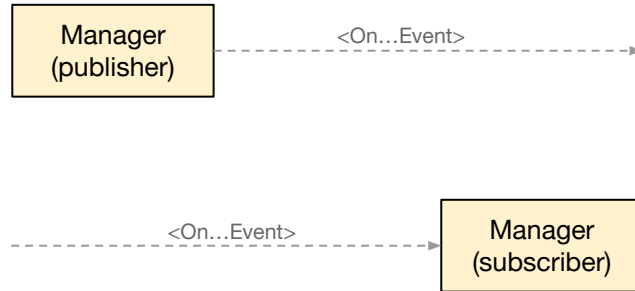
Note: the receiving manager should not queue commands back to the sending manager.

Actual



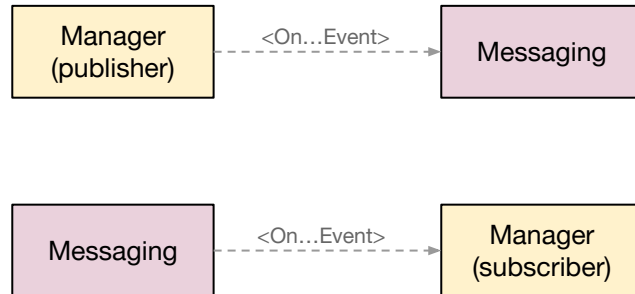
Pub/Sub

Pattern



A manager can publish events that one or more managers can subscribe to.

Actual



Timer

Pattern



Actual

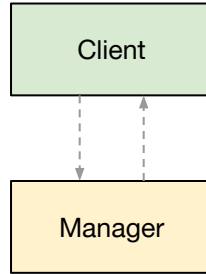


The Timer functions by delivering messages via Messaging based on a schedule. This is useful for kicking off batch processes at regular intervals.

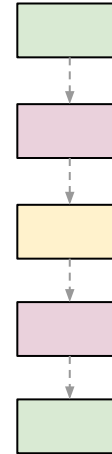
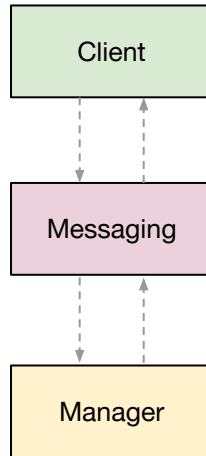


Client Async Interaction

Pattern



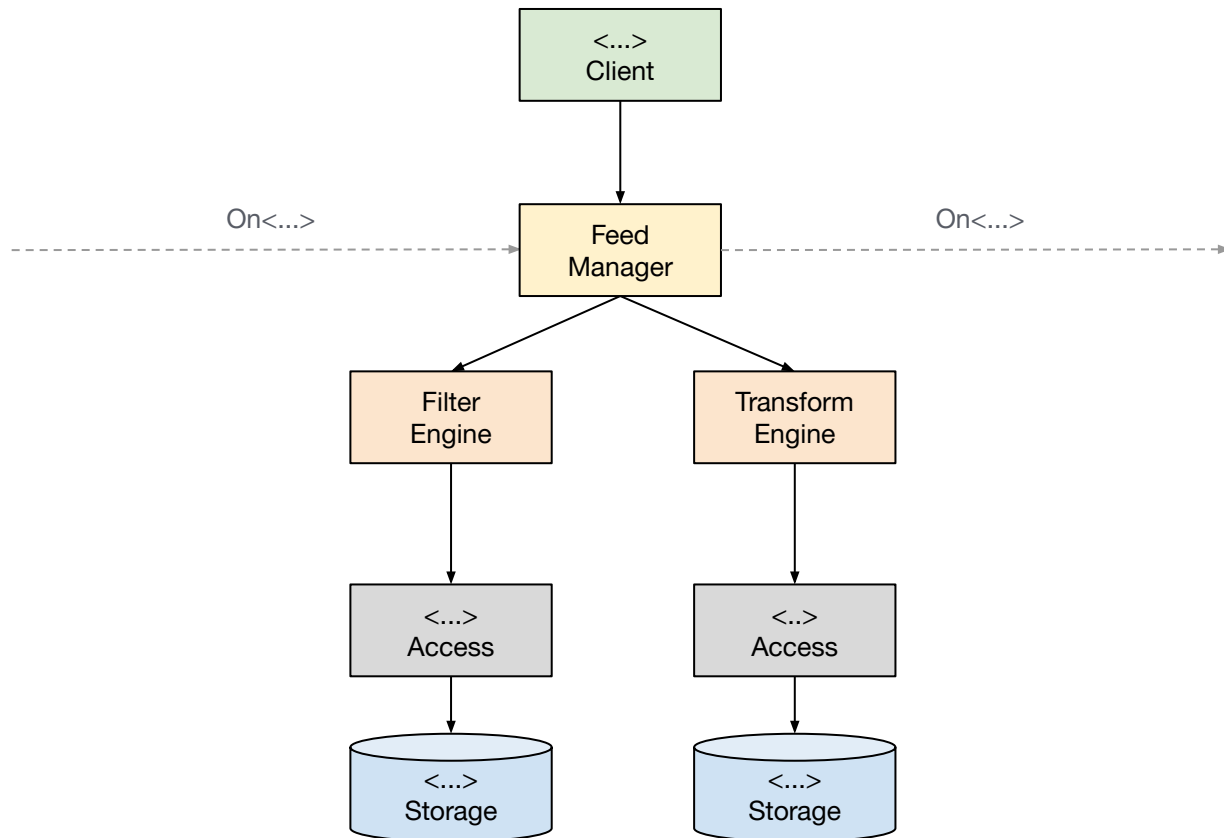
Actual



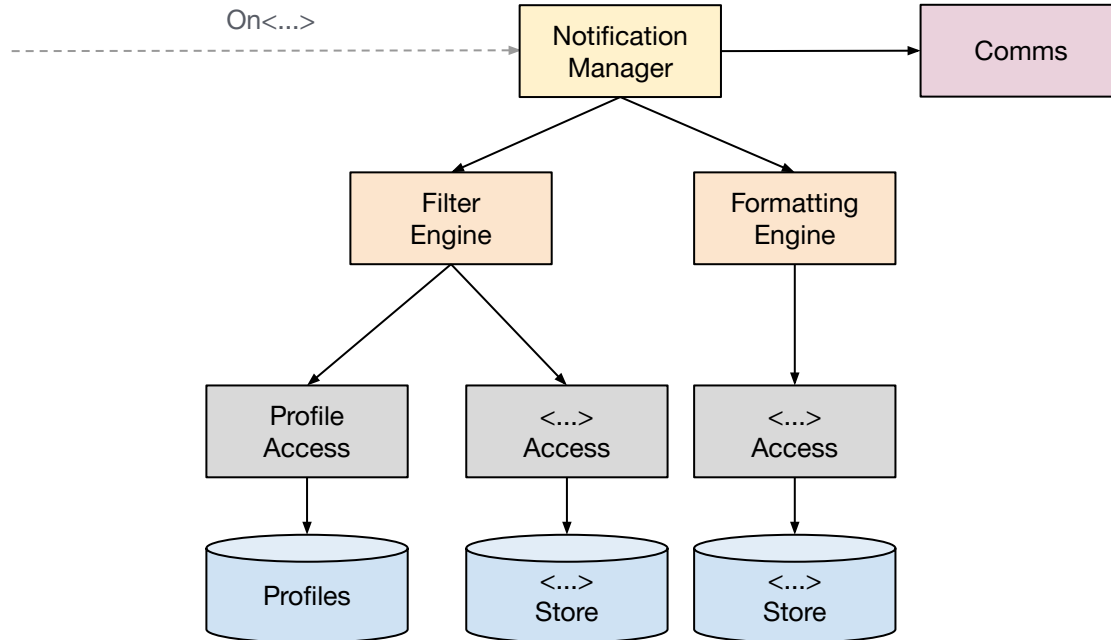
When a message is sent asynchronously from a client to a manager or vice versa, the existence of the Message Bus Utility is implied.



Ingest/Digest



Notify



Notification preferences are stored in Profiles.

Filter determines the appropriate notification Channel based on preferences and configuration.

Formatting formats the notification for the appropriate channel(s)

Comms encapsulates the specific Email or SMS functionality.

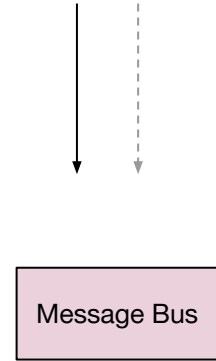
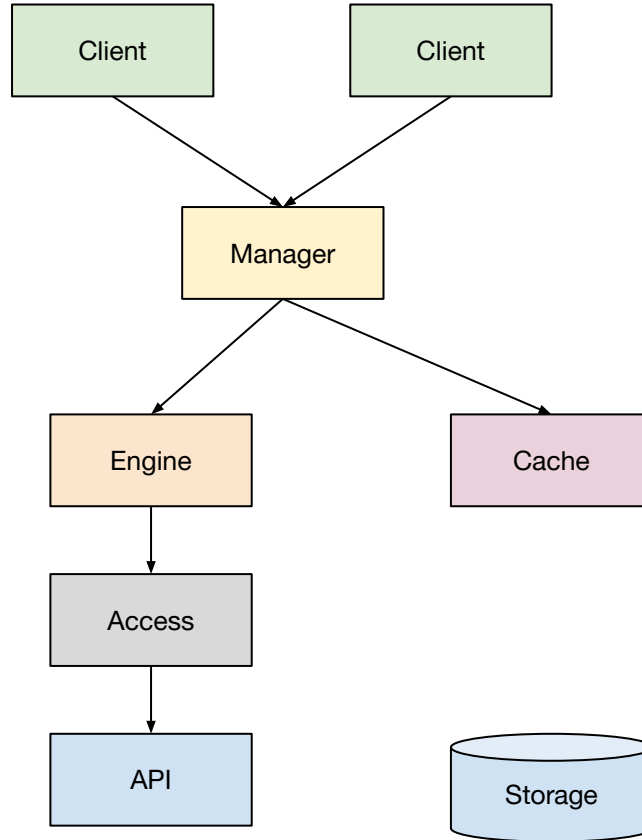


Call Chains



{use case name}

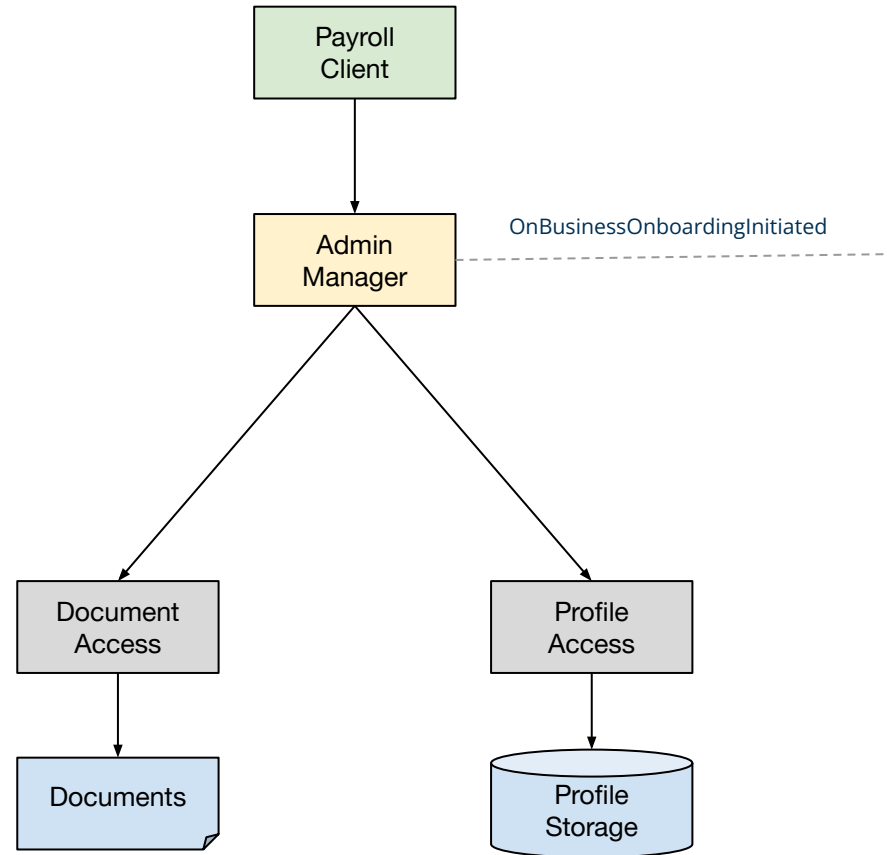
An easy to use palette is good to have...



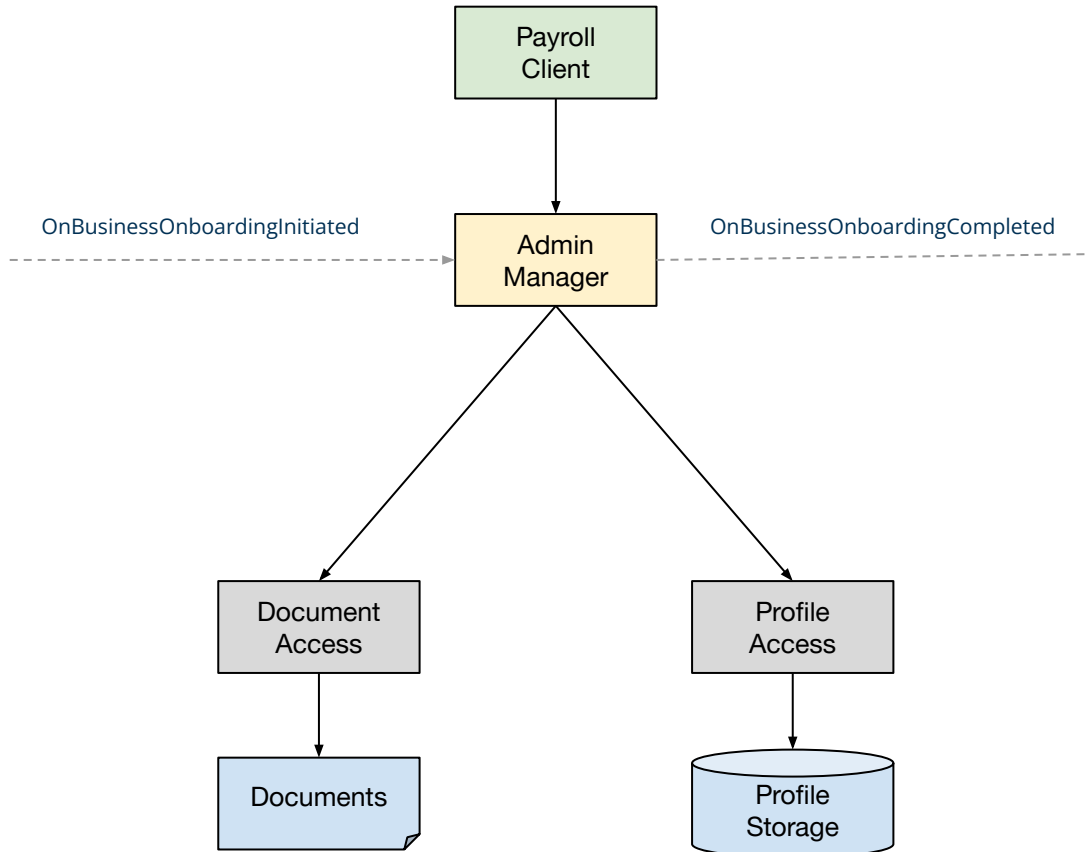
Admin



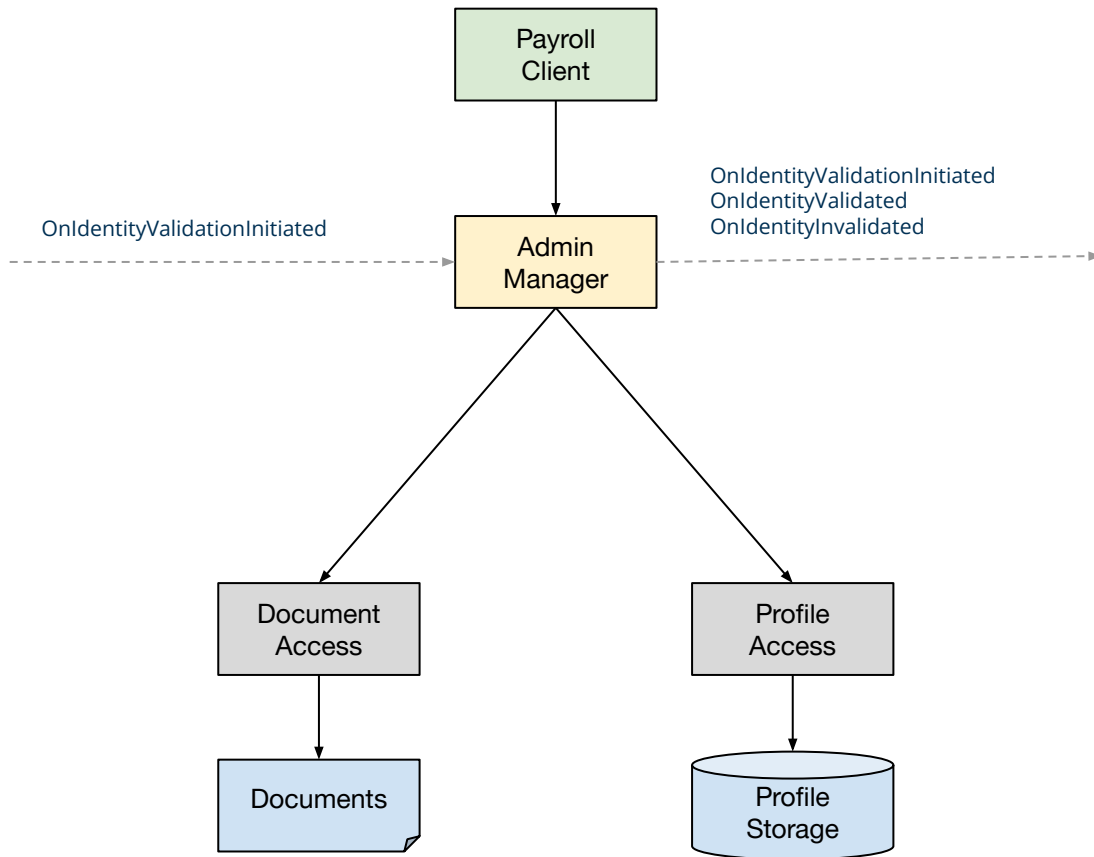
Onboard Business



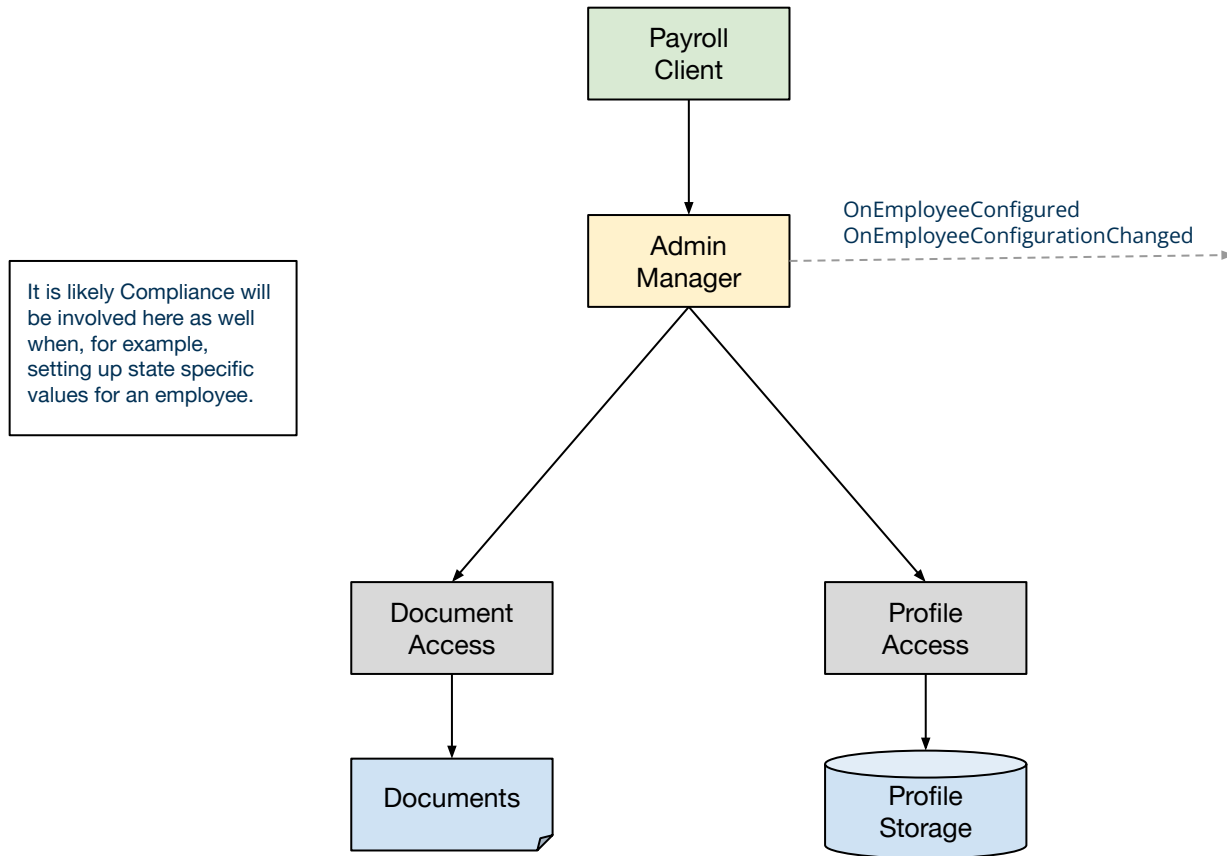
Validate Business



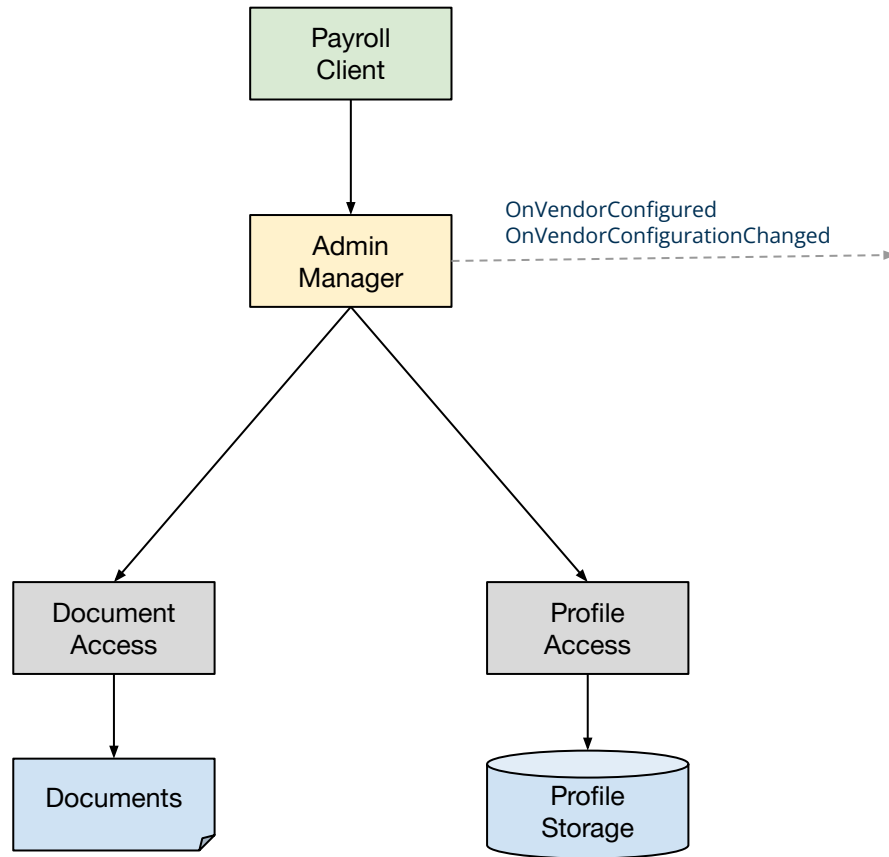
Validate Identity



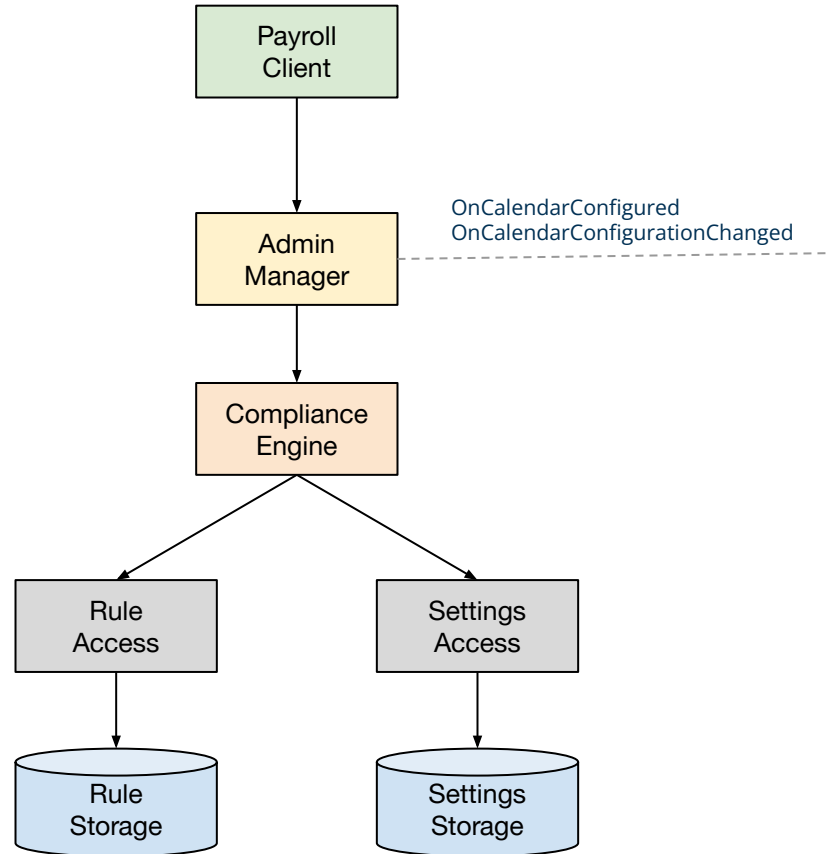
Configure Employee



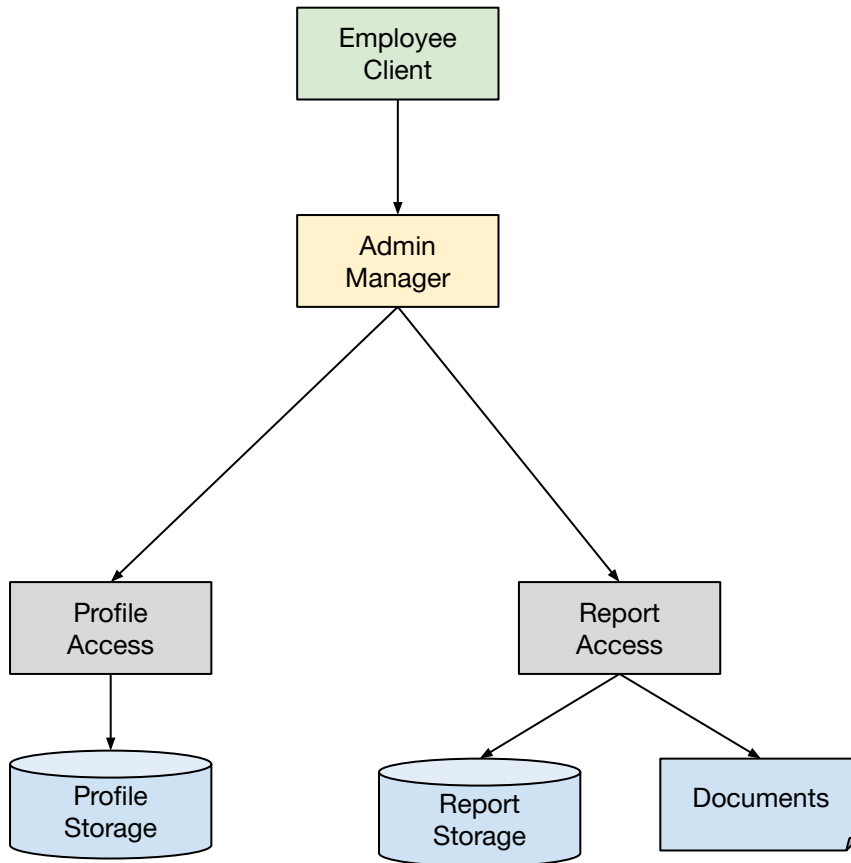
Configure Vendor



Configure Calendar



Review Paystubs



Collect



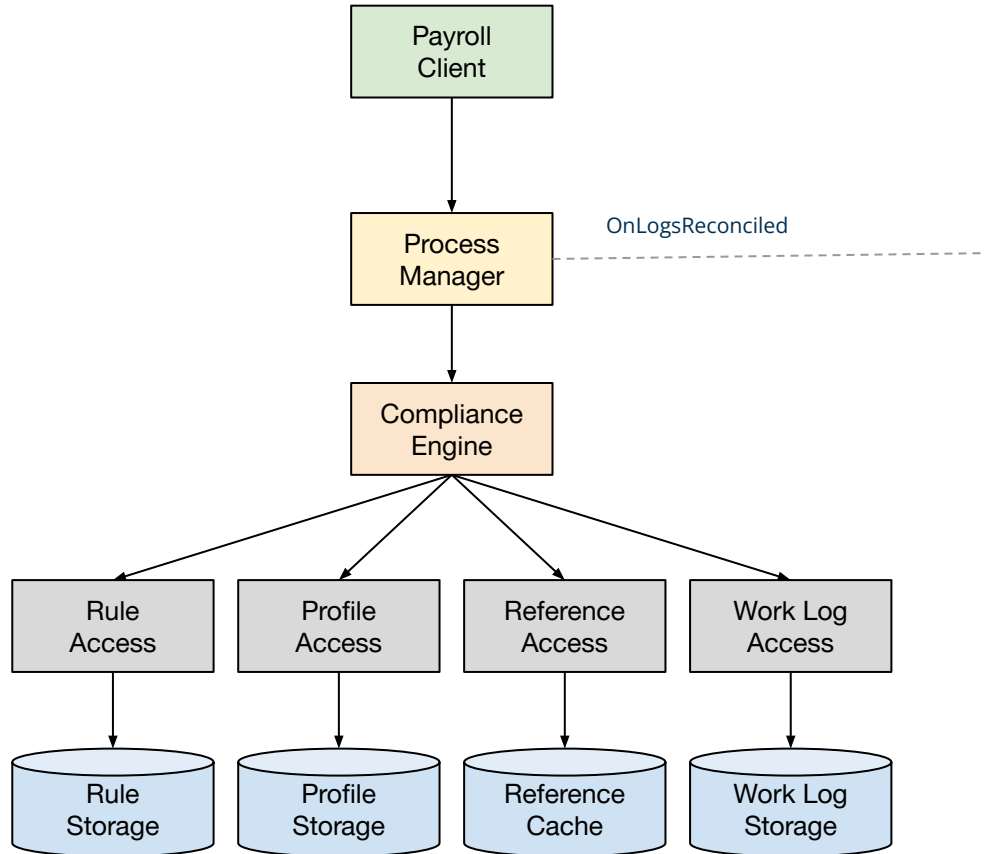
(Traqspera?)



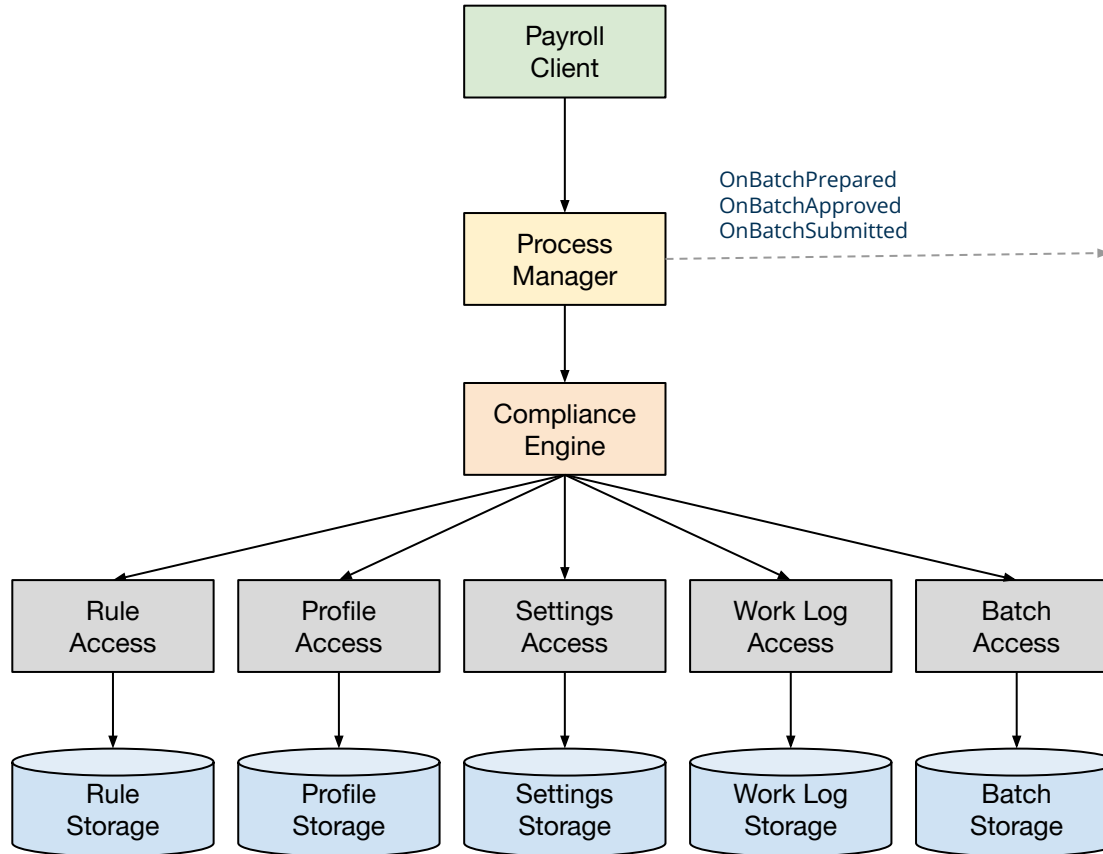
Process



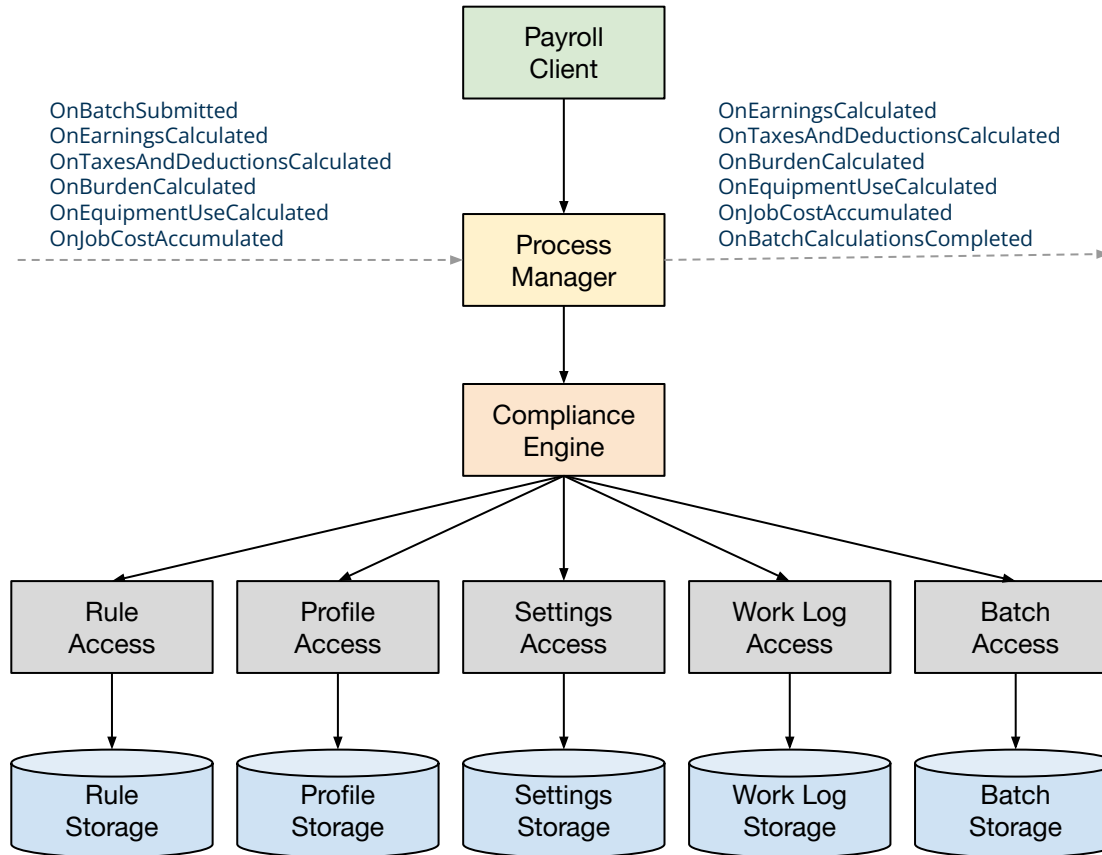
Reconcile Logs



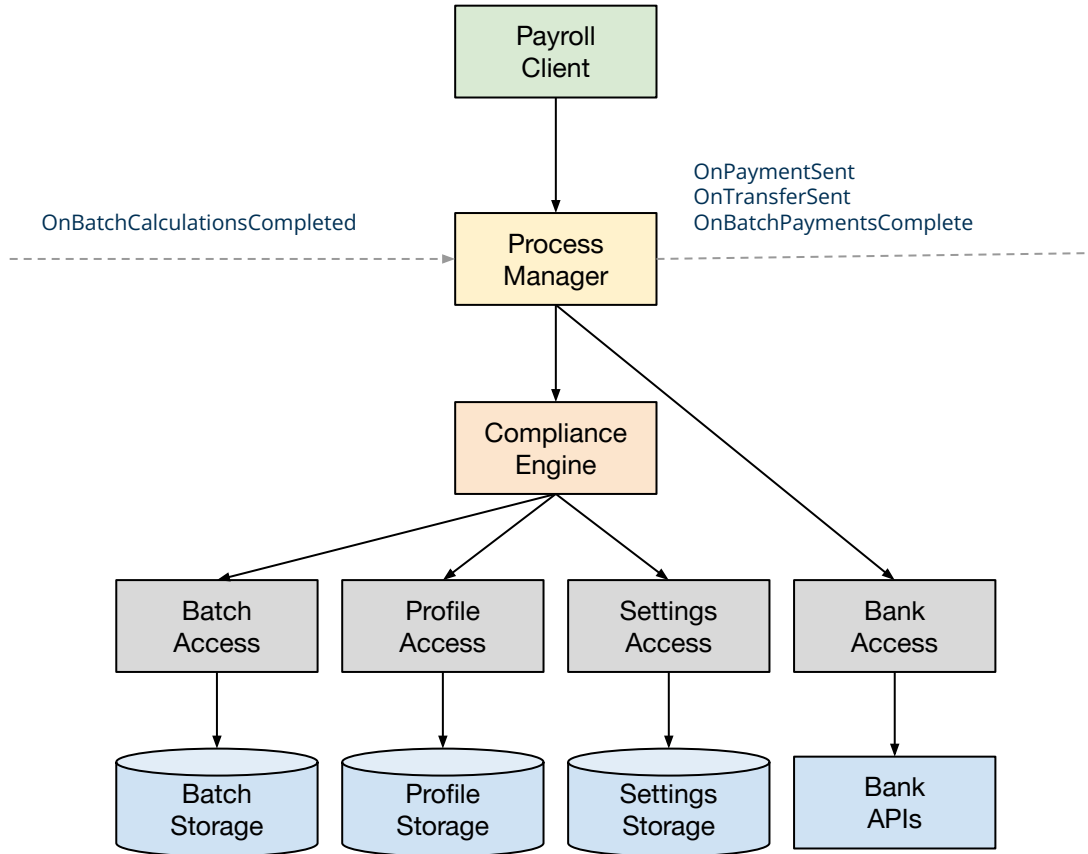
Prepare/Submit Batch



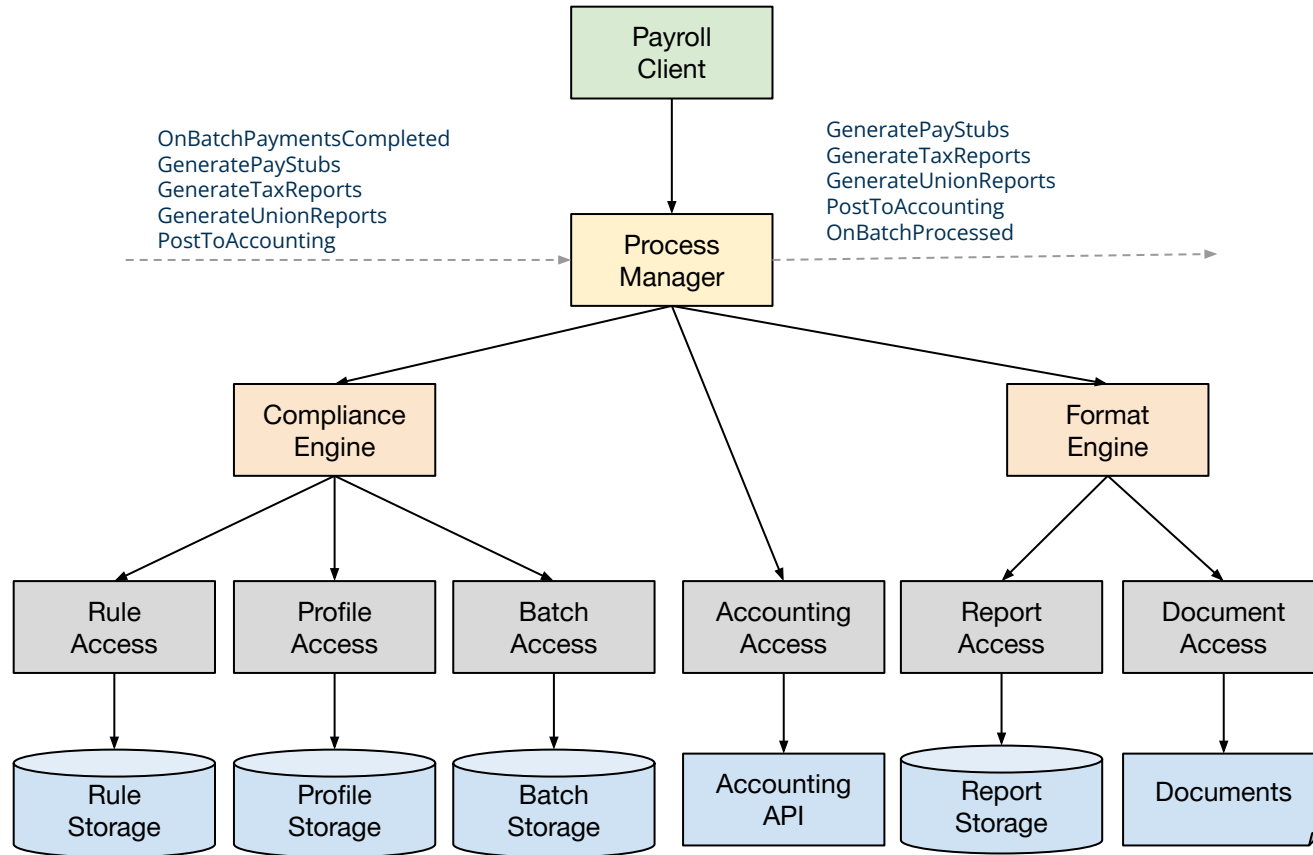
Process Batch (Calculate)



Process Batch (Pay/Transfer)



Process Batch (Report)

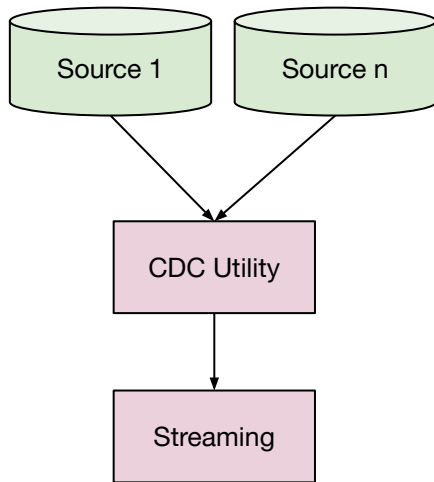


Feed

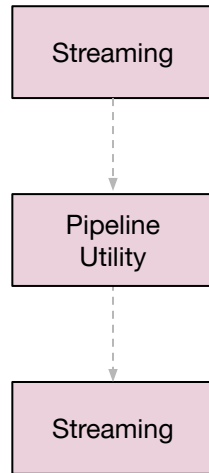


Stream

Source



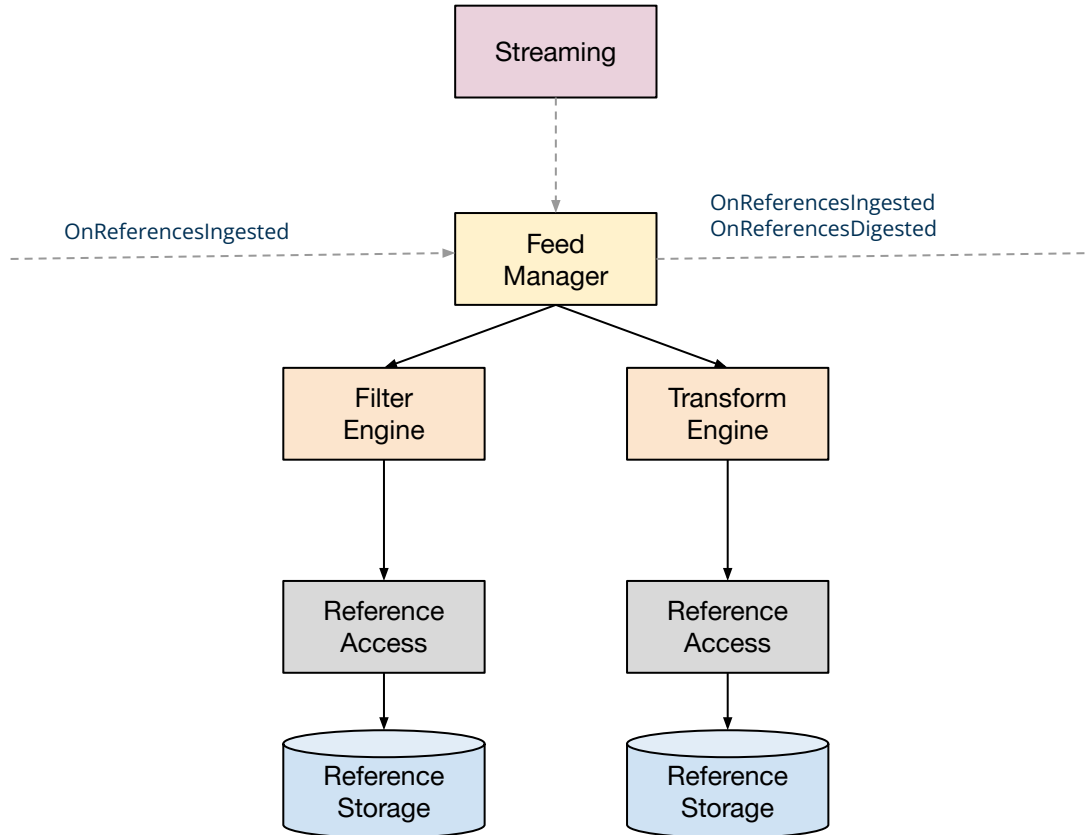
Process



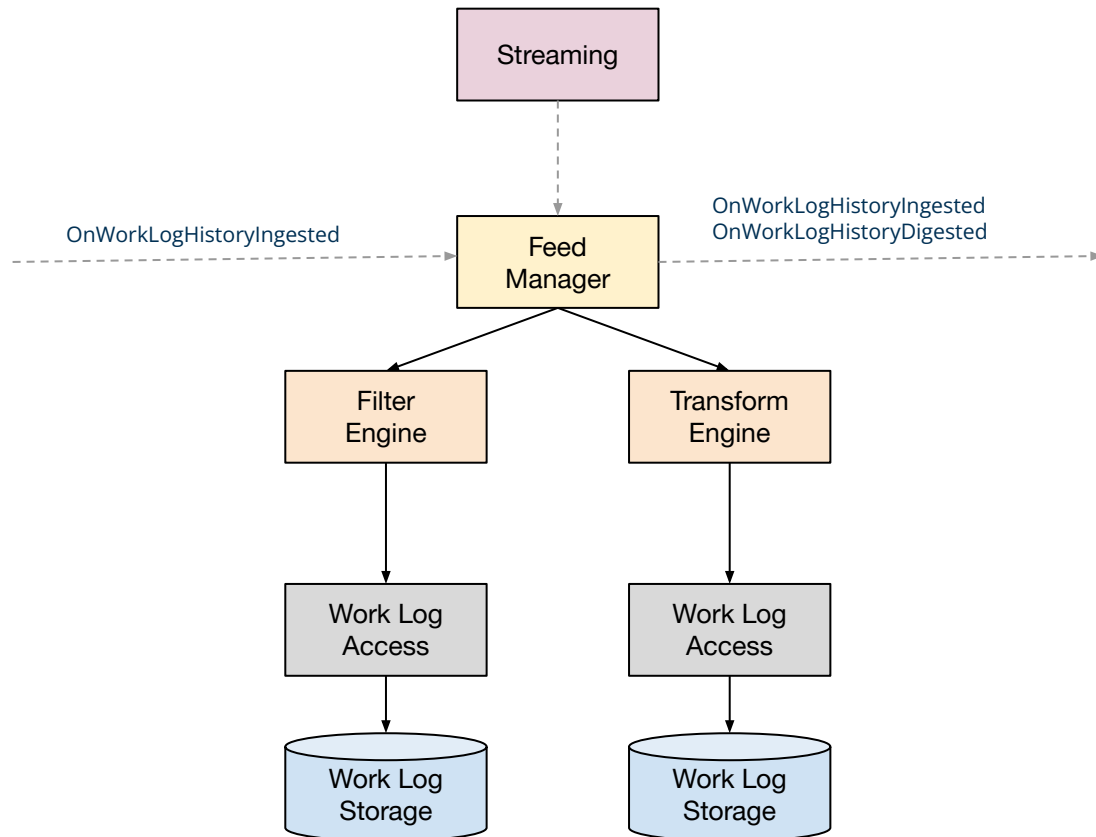
Consume



Ingest/Digest References



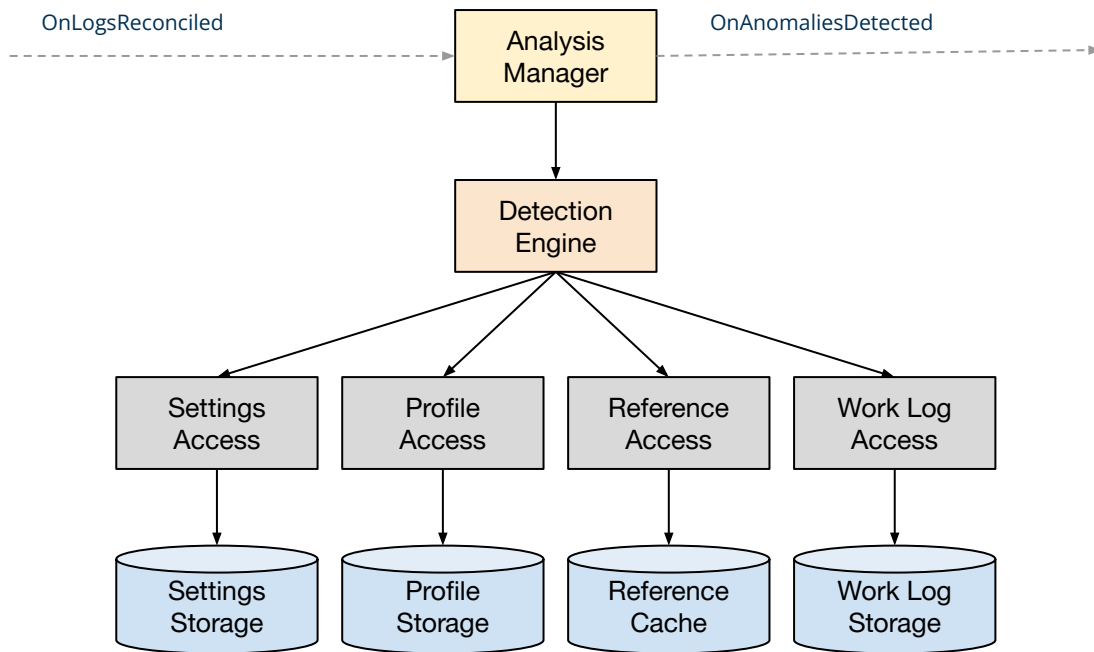
Ingest/Digest Work Log History



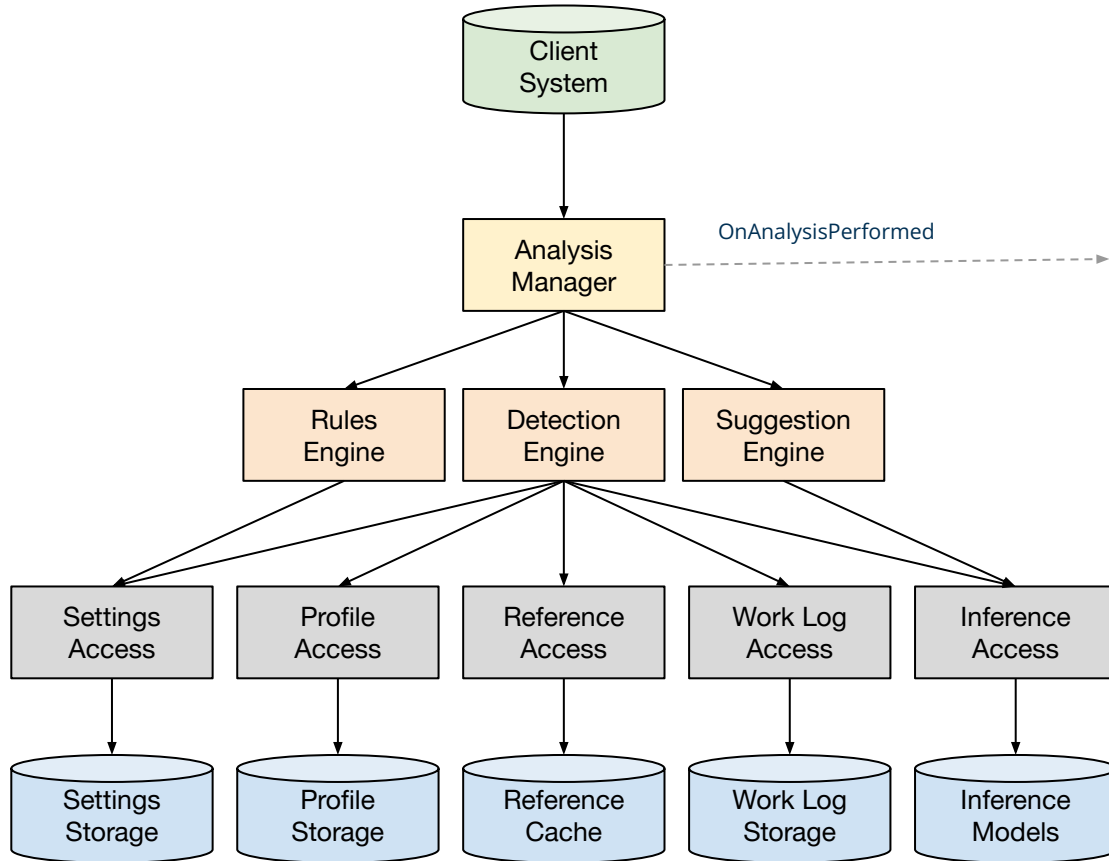
Analyze



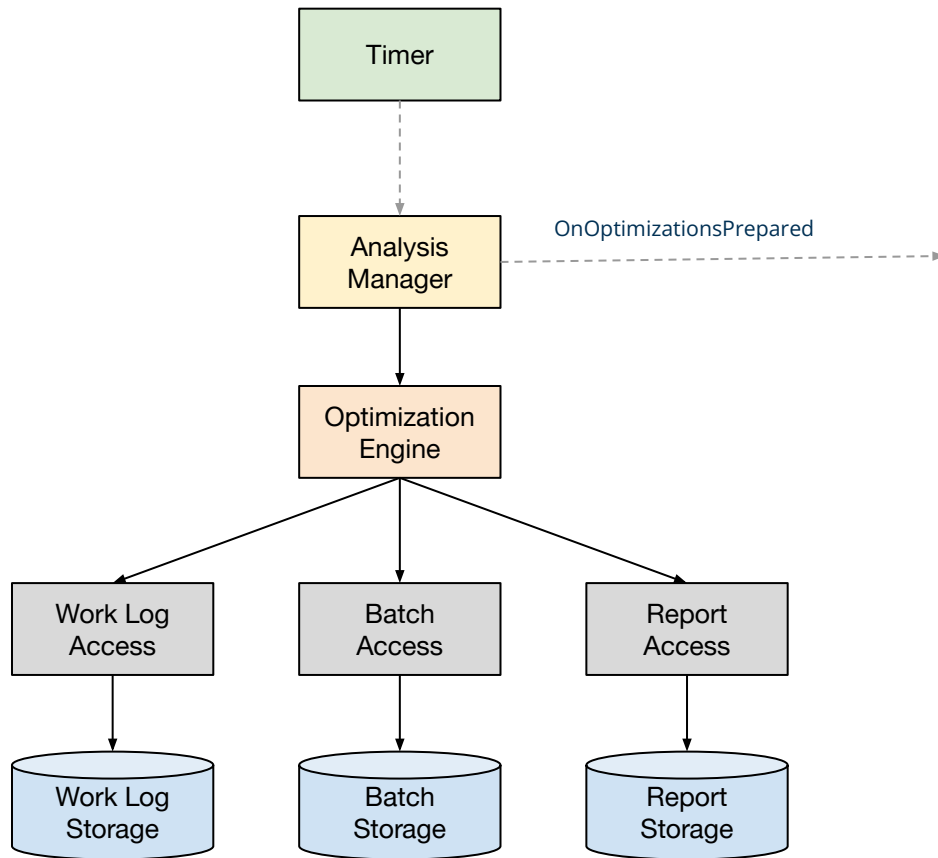
Detect Anomalies



Perform Analysis



Optimize



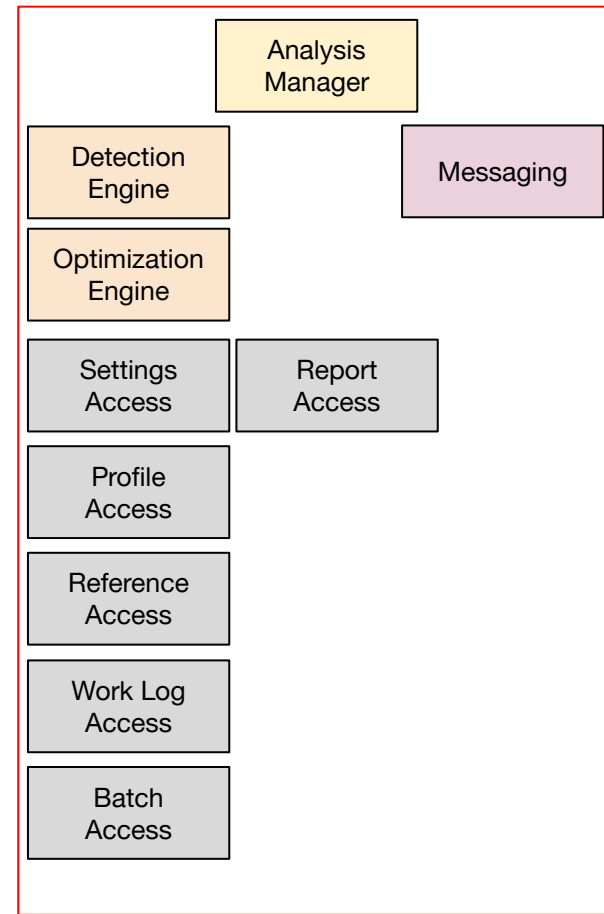
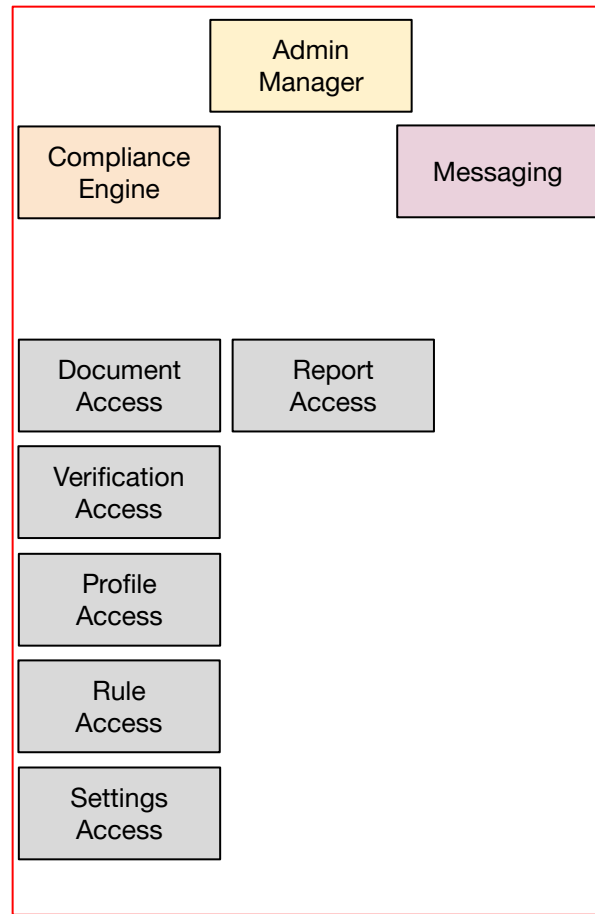
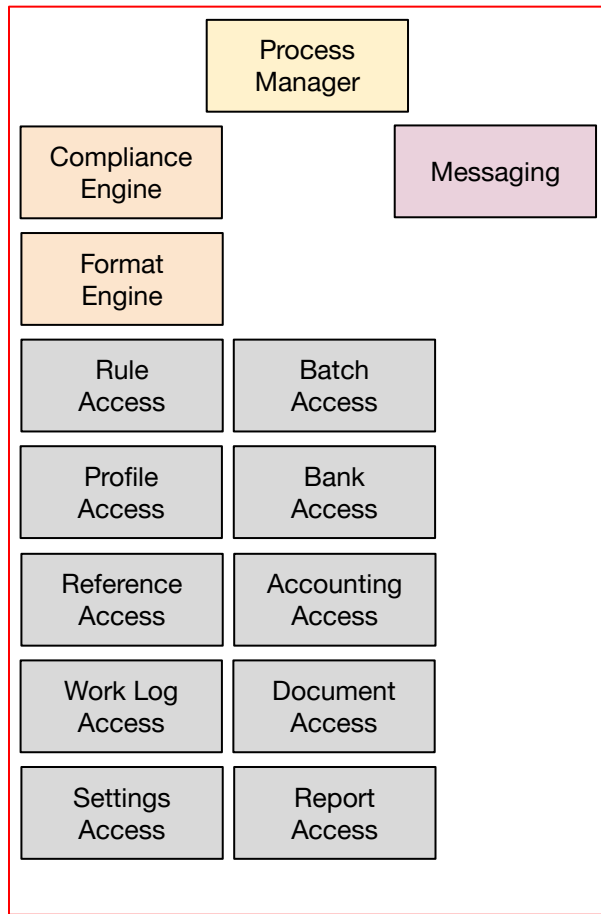
Suggest

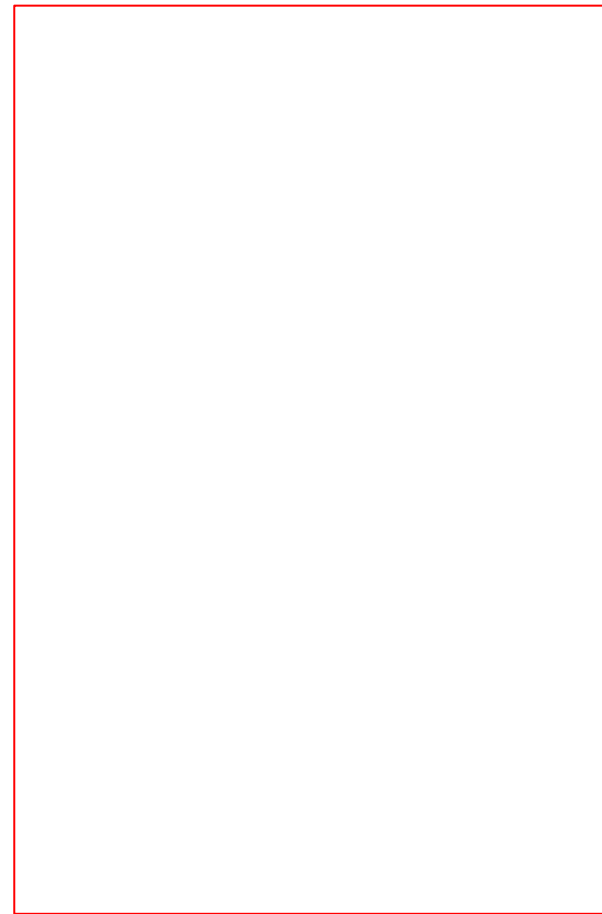
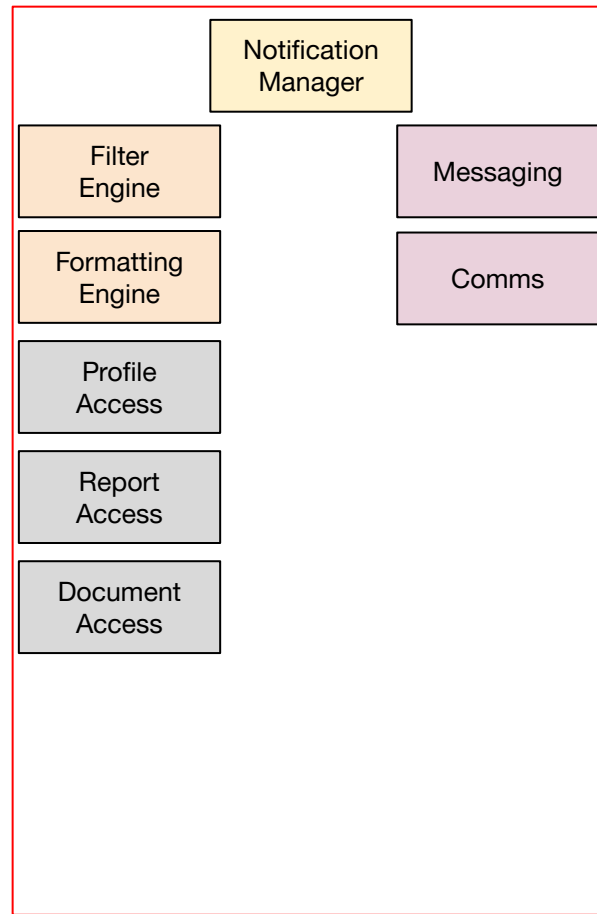
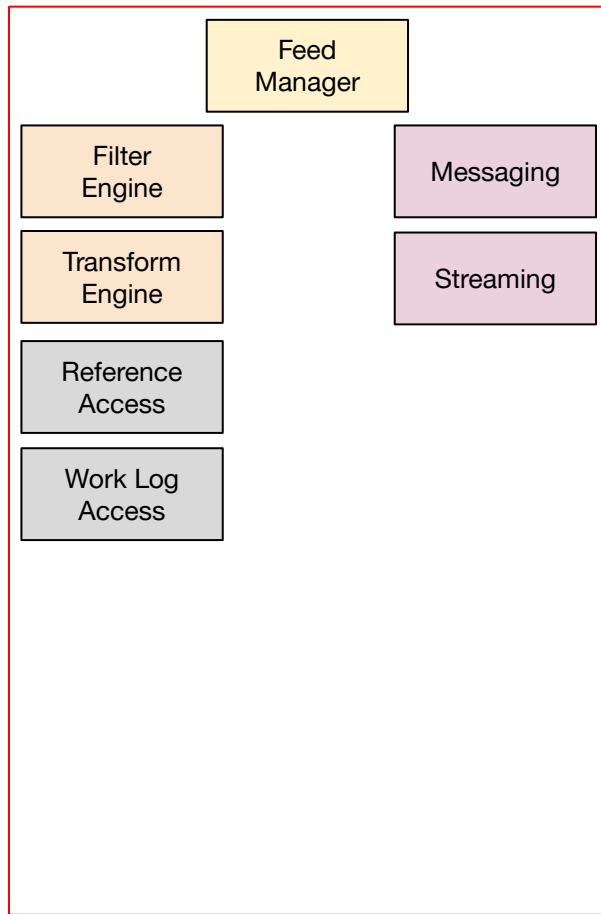
TBD



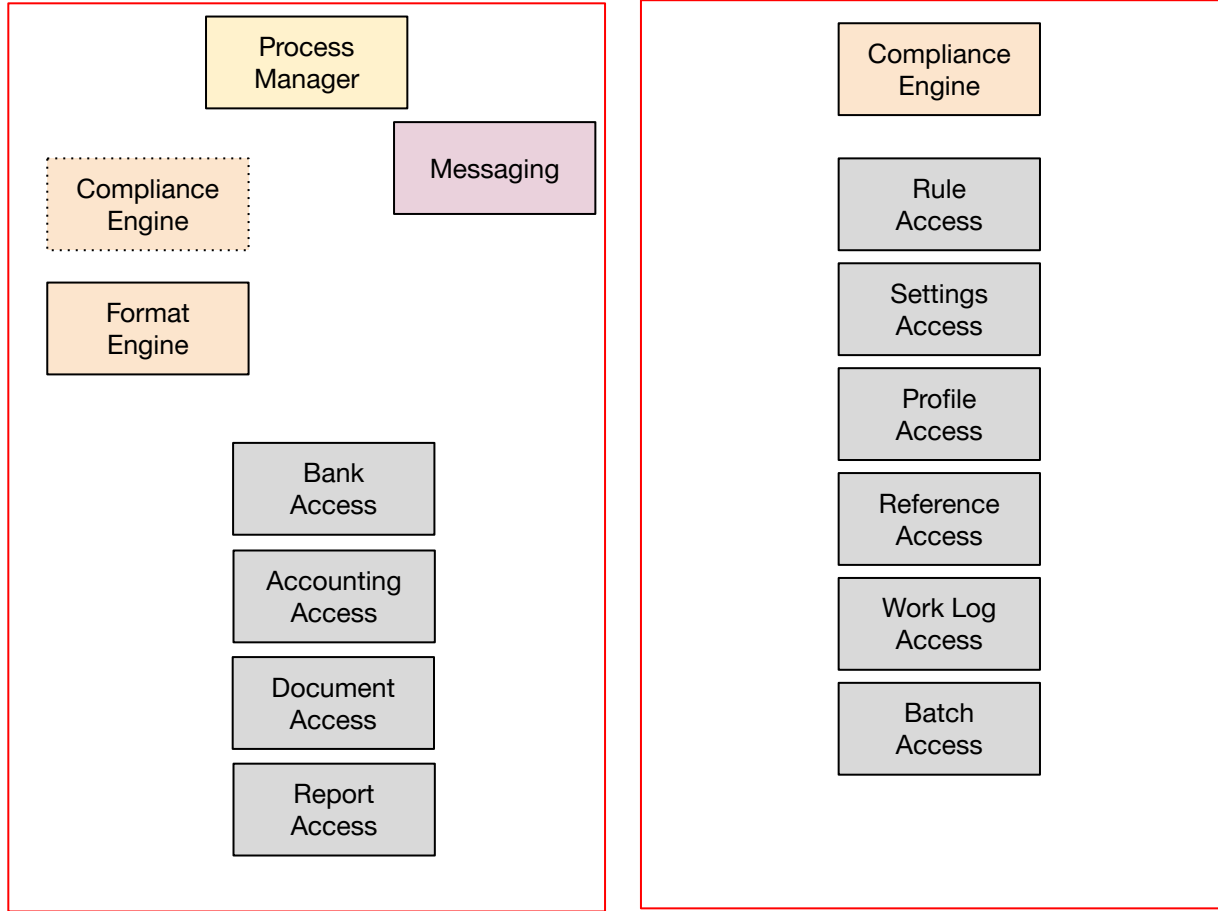
Services







Due to the significant resource contention for compliance processing, this alternative logical service composition may make more sense than the standard approach of one subsystem per manager.



Compliance Microservice Concept

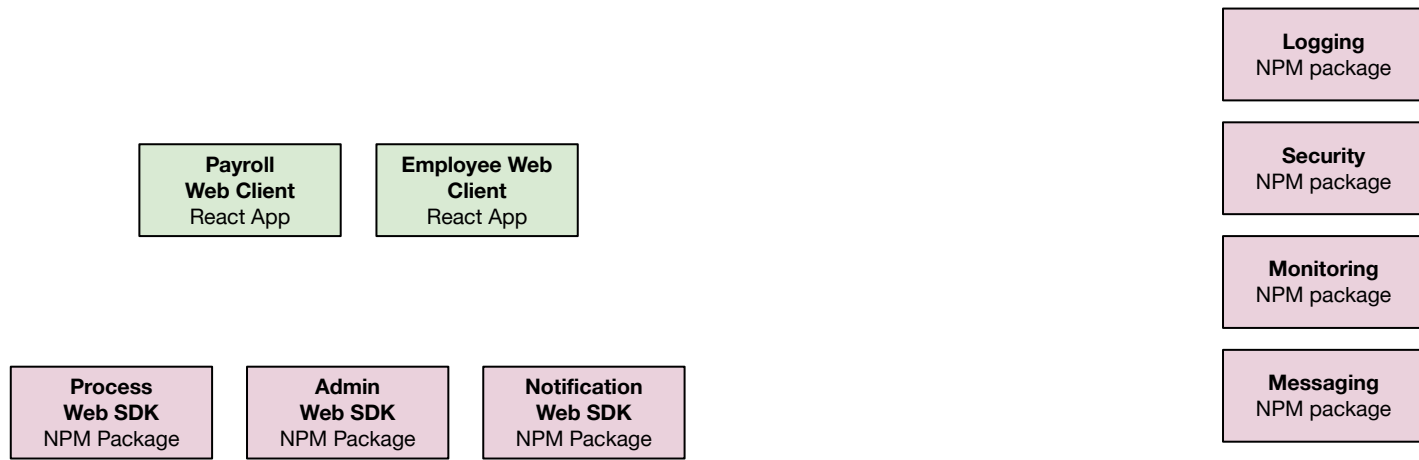


Packages



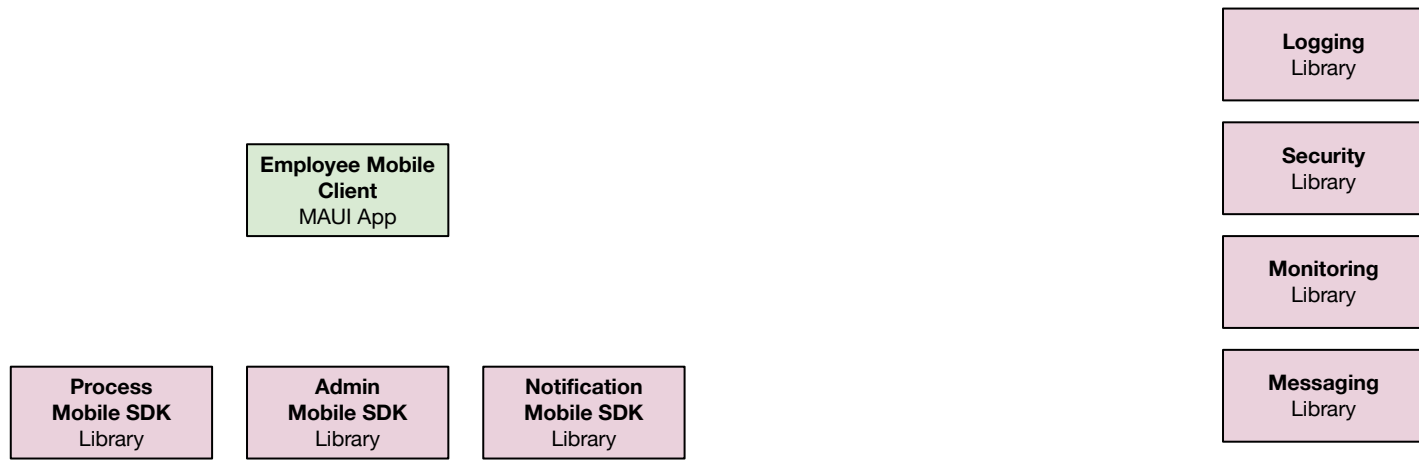
Web

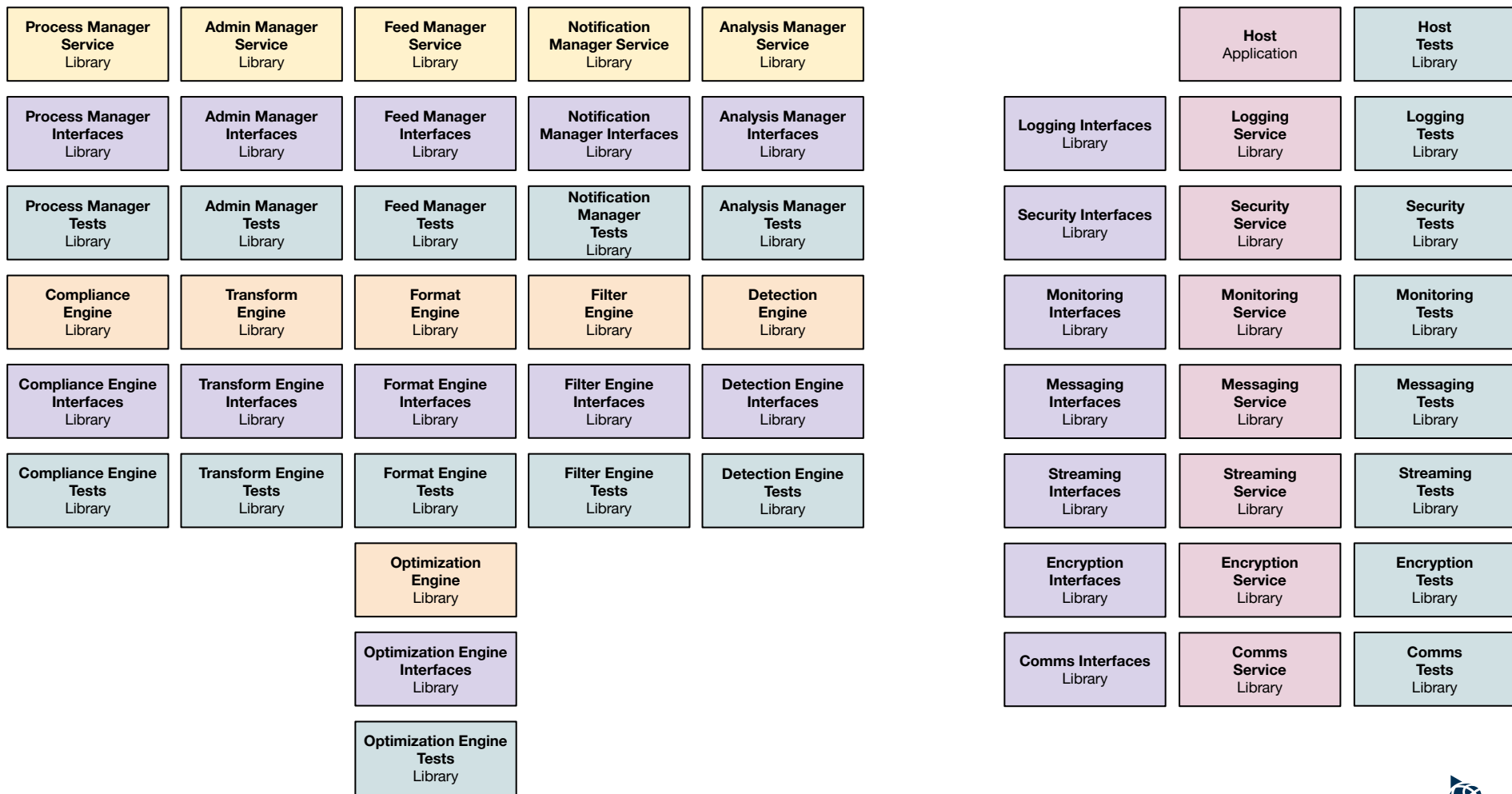




Web Client Package Allocation





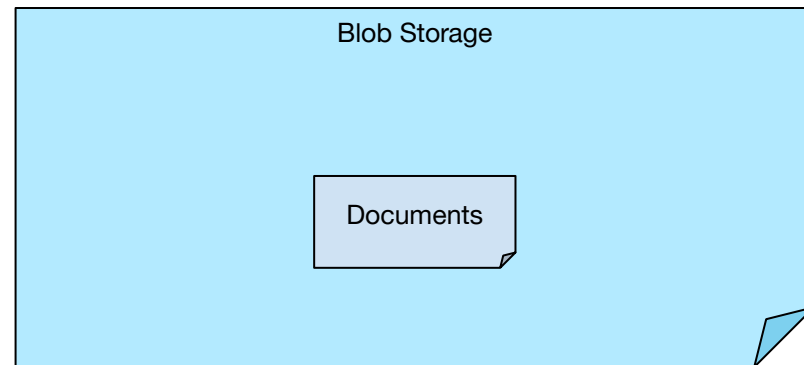
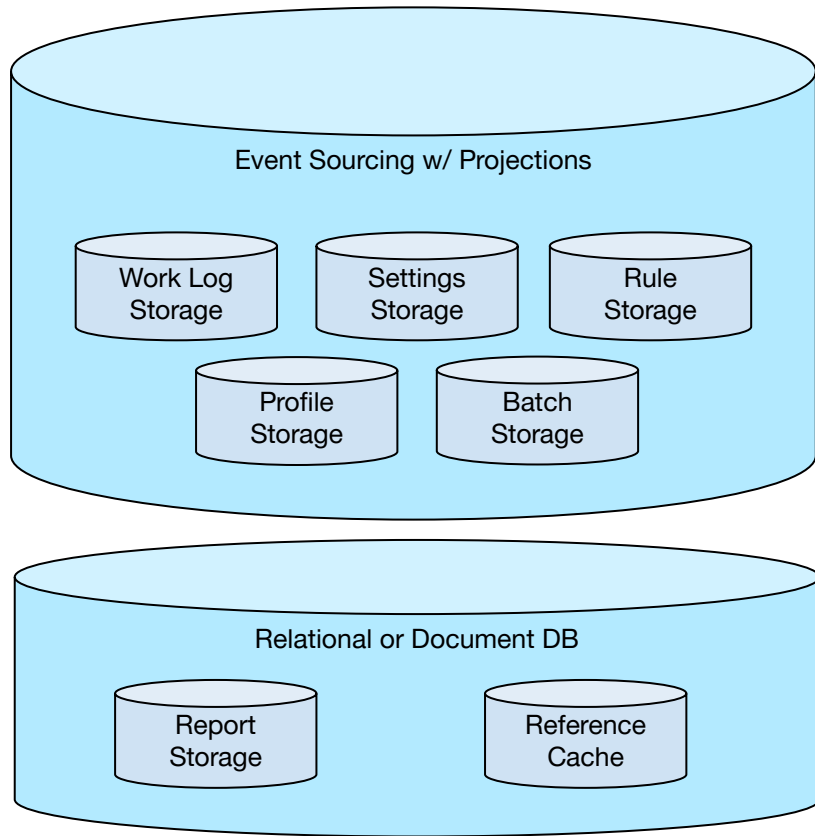


Server-side assembly allocation



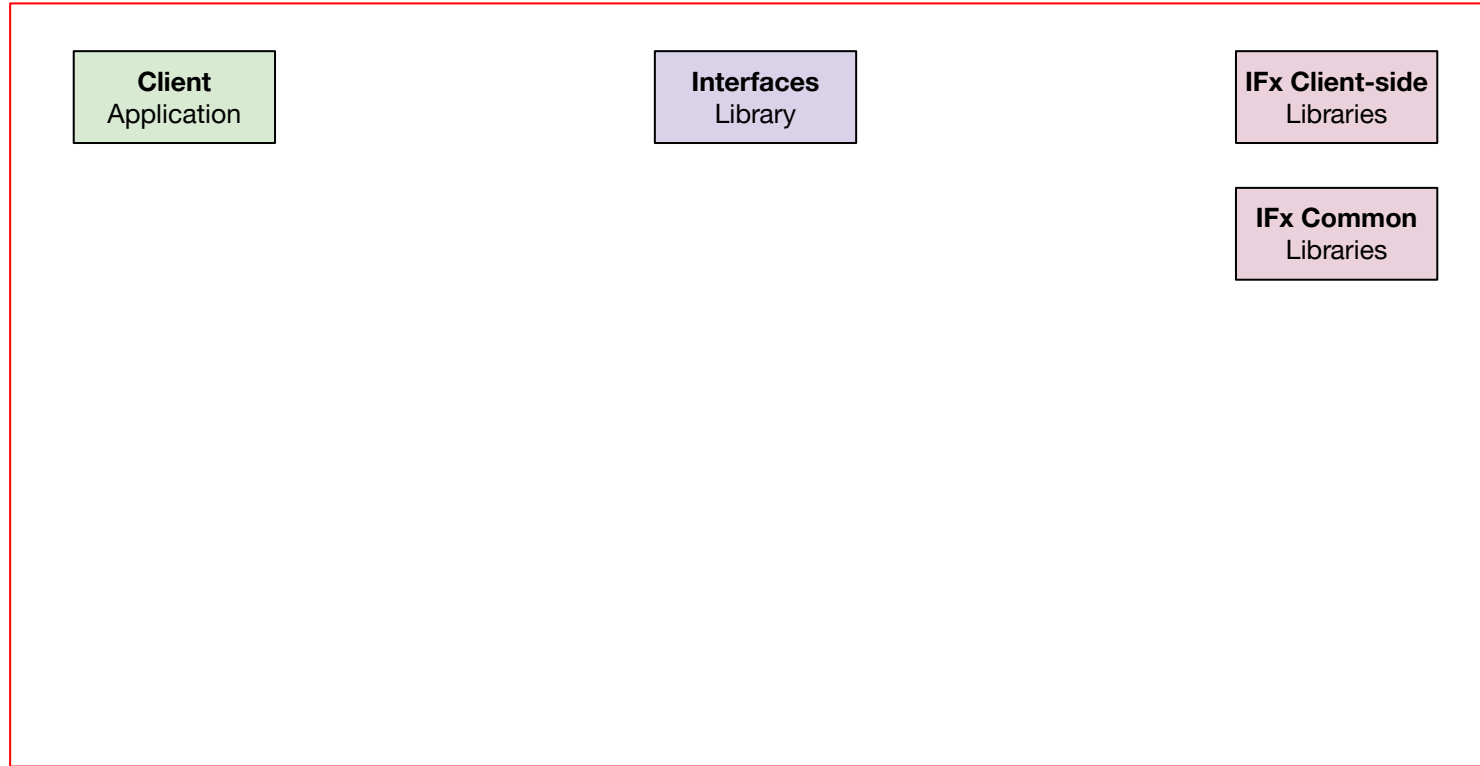
Work Log Access Service Library	Profile Access Service Library	Settings Access Service Library	Batch Access Service Library	Rule Access Service Library	Reference Access Service Library	Accounting Access Service Library	Verification Access Service Library
Work Log Access Interfaces Library	Profile Access Interfaces Library	Settings Access Interfaces Library	Batch Access Interfaces Library	Rule Access Interfaces Library	Reference Access Interfaces Library	Accounting Access Interfaces Library	Verification Access Interfaces Library
Work Log Access Tests Library	Profile Access Tests Library	Settings Access Tests Library	Batch Access Tests Library	Rule Access Tests Library	Reference Access Tests Library	Accounting Access Tests Library	Verification Access Tests Library
Report Access Service Library	Bank Access Service Library	Document Access Service Library					
Report Access Interfaces Library	Bank Access Interfaces Library	Document Access Interfaces Library					
Report Access Tests Library	Bank Access Tests Library	Document Access Tests Library					

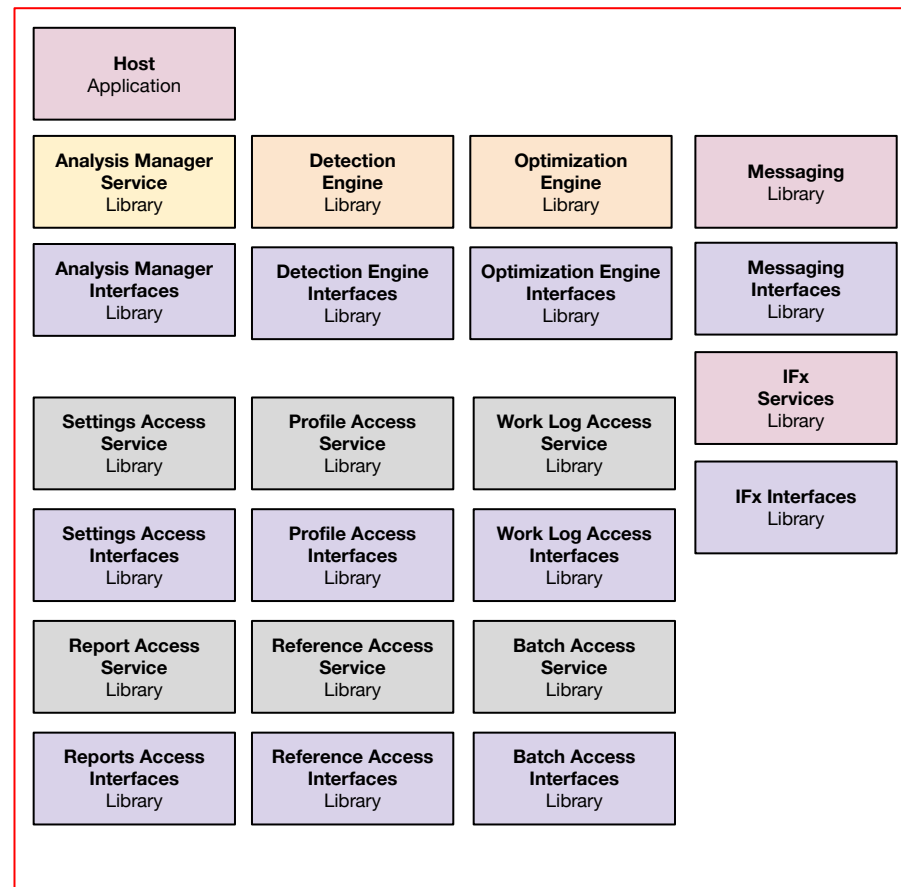
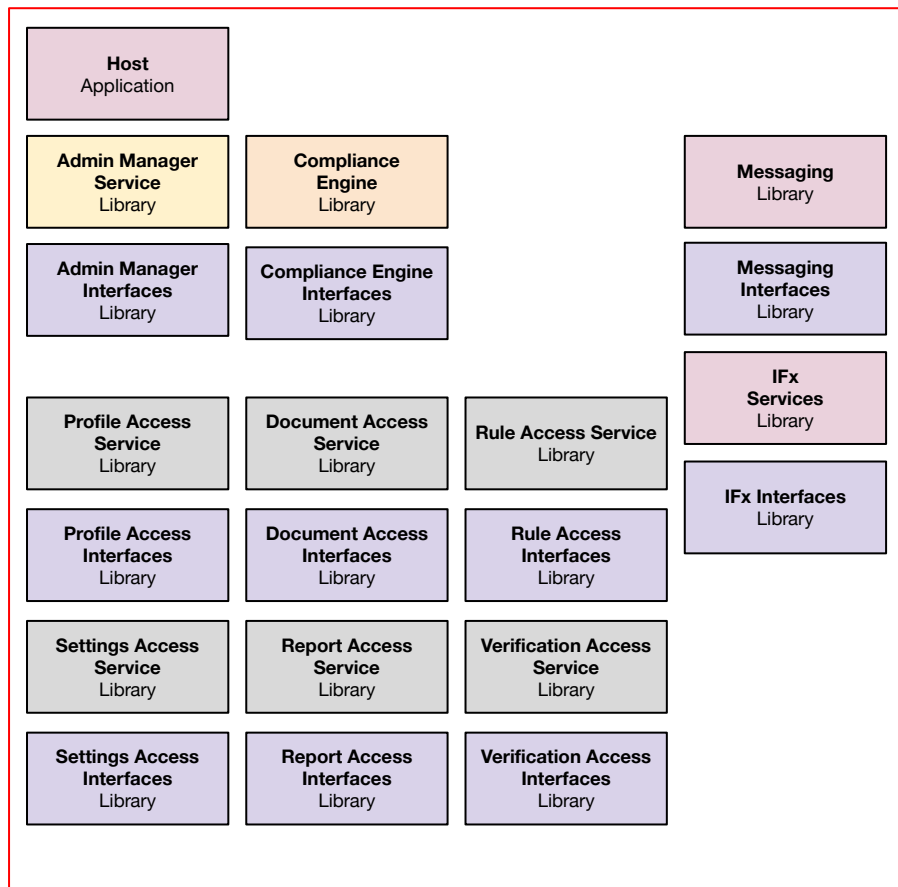




Deployment

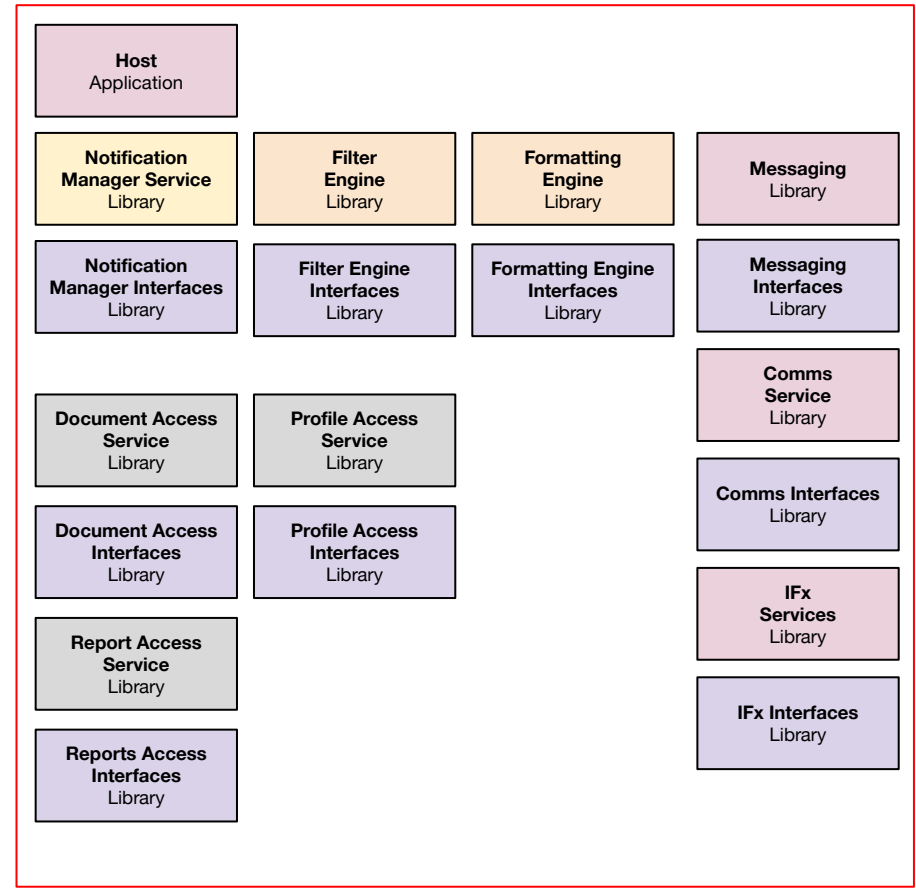
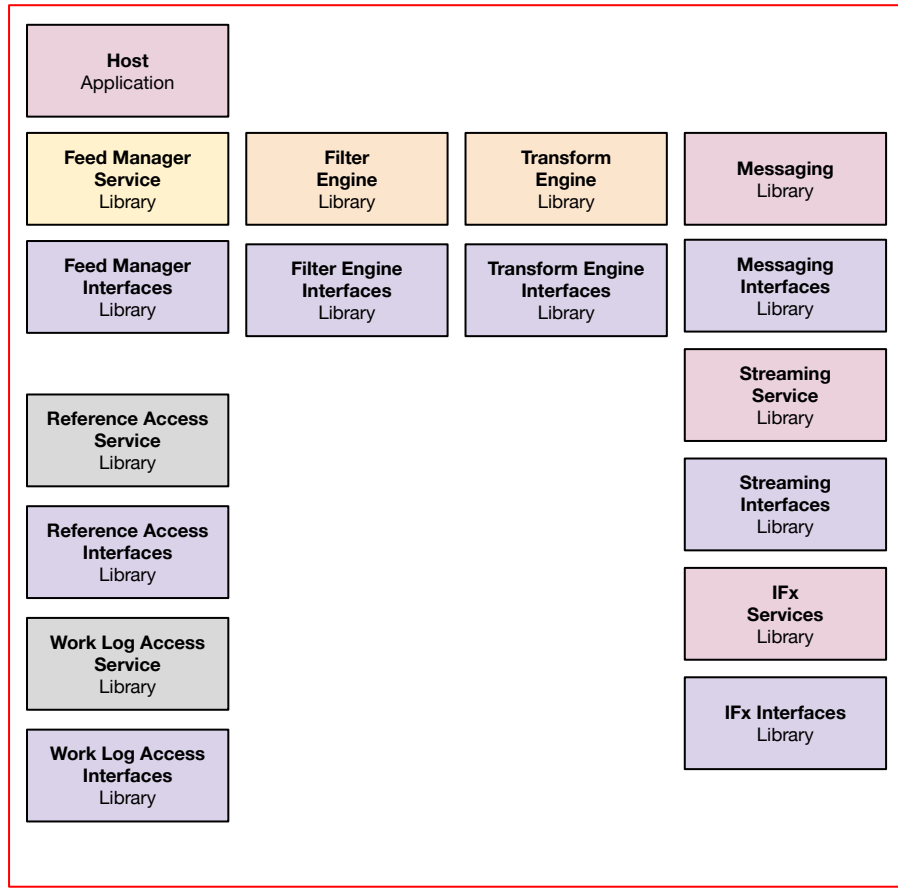


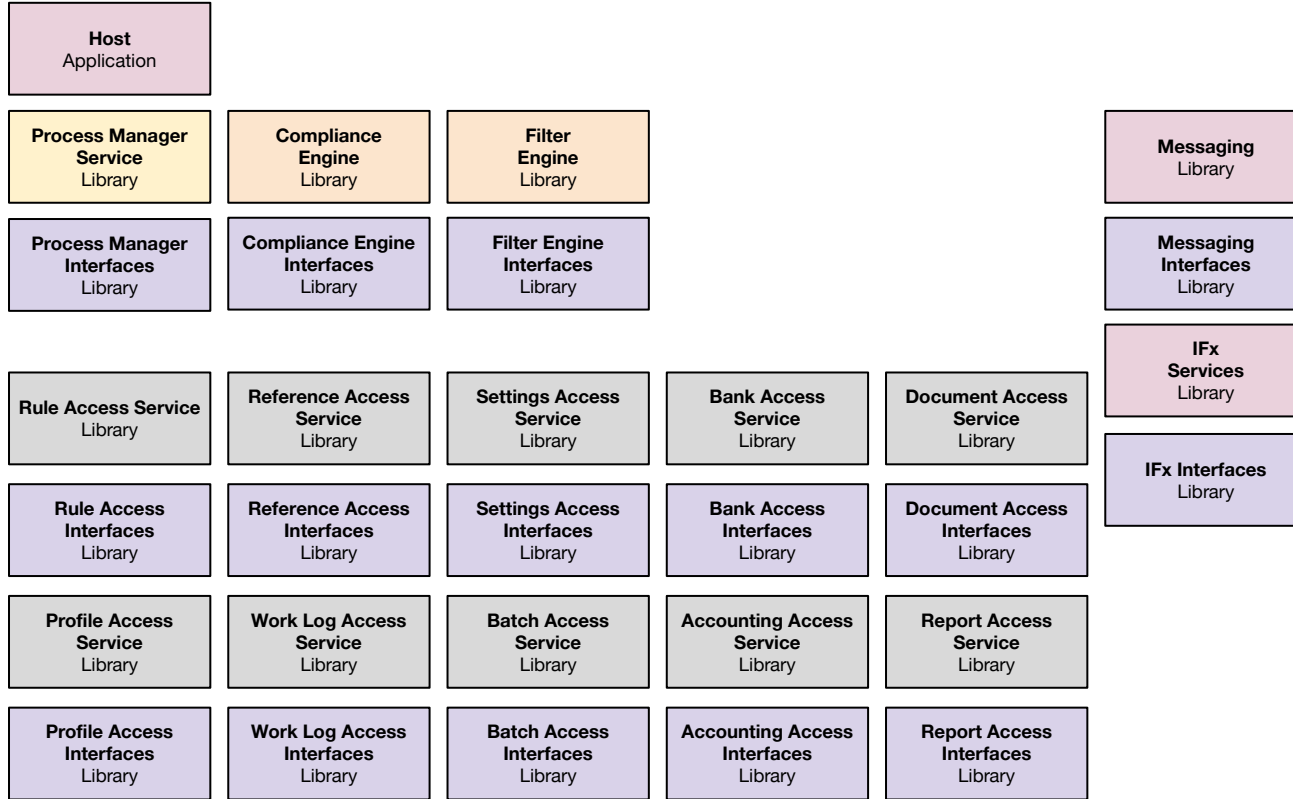




Physical Deployment - Cloud System (Admin and Analysis)







Physical Deployment - Cloud System (Process)





Vertical Slice



Client

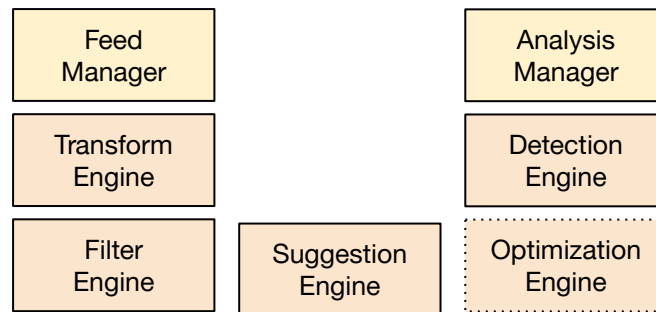


Vertical Slice

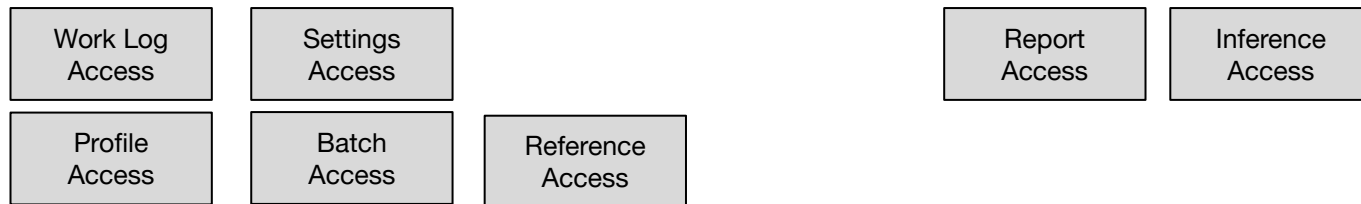
Utilities



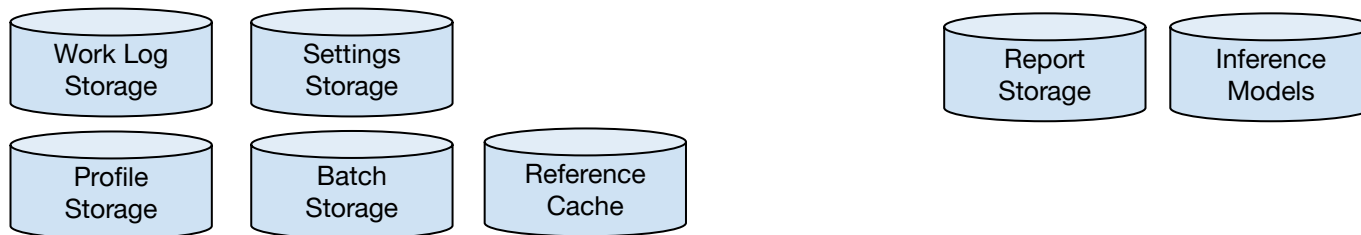
Logic



Access



Resource



Authentication

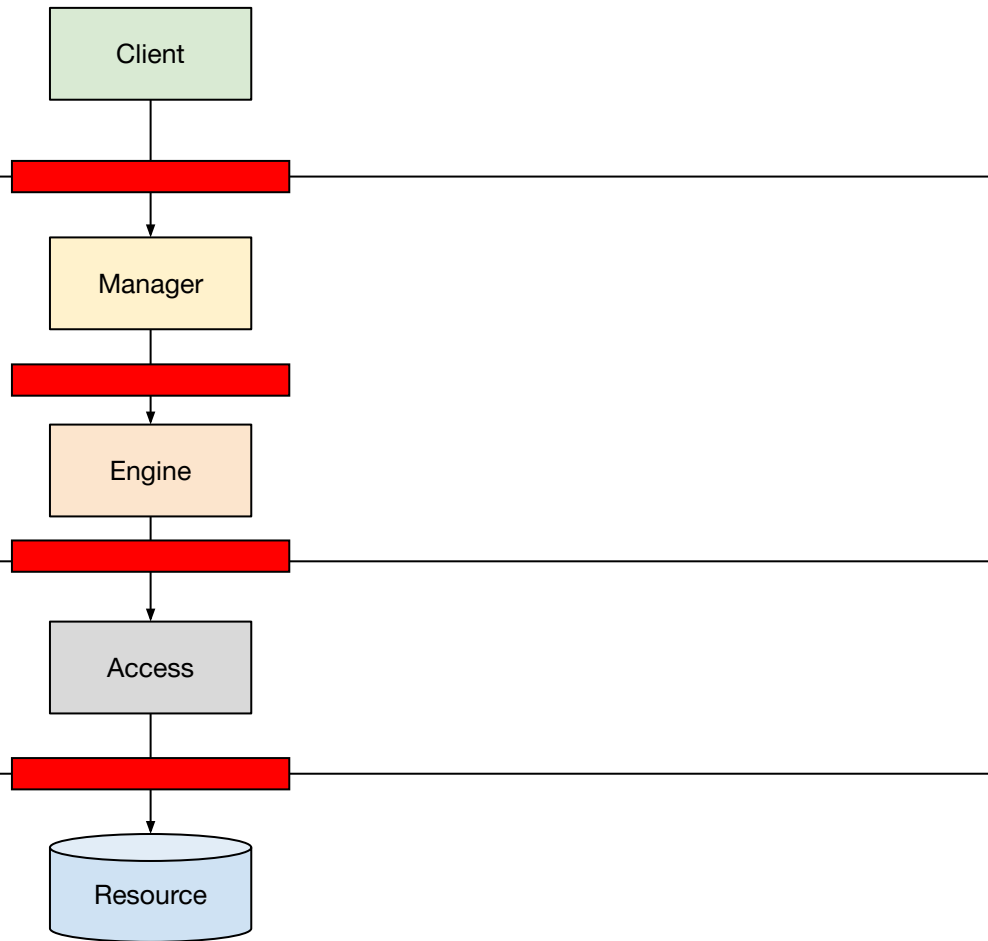


Client

Logic

Access

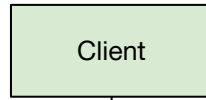
Resource



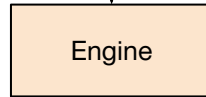
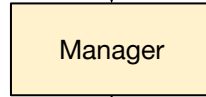
Authentication - Best Practice



Client



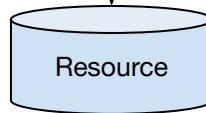
Logic



Access



Resource



Authentication - Pragmatic Intranet



Authentication



Patterns

Common Authorization Zones

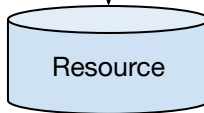
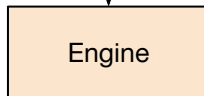
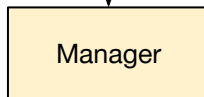


Client

Logic

Access

Resource



Note: apply authorization on operations at Manager level preferably via custom attributes.

Authorization - Use Case

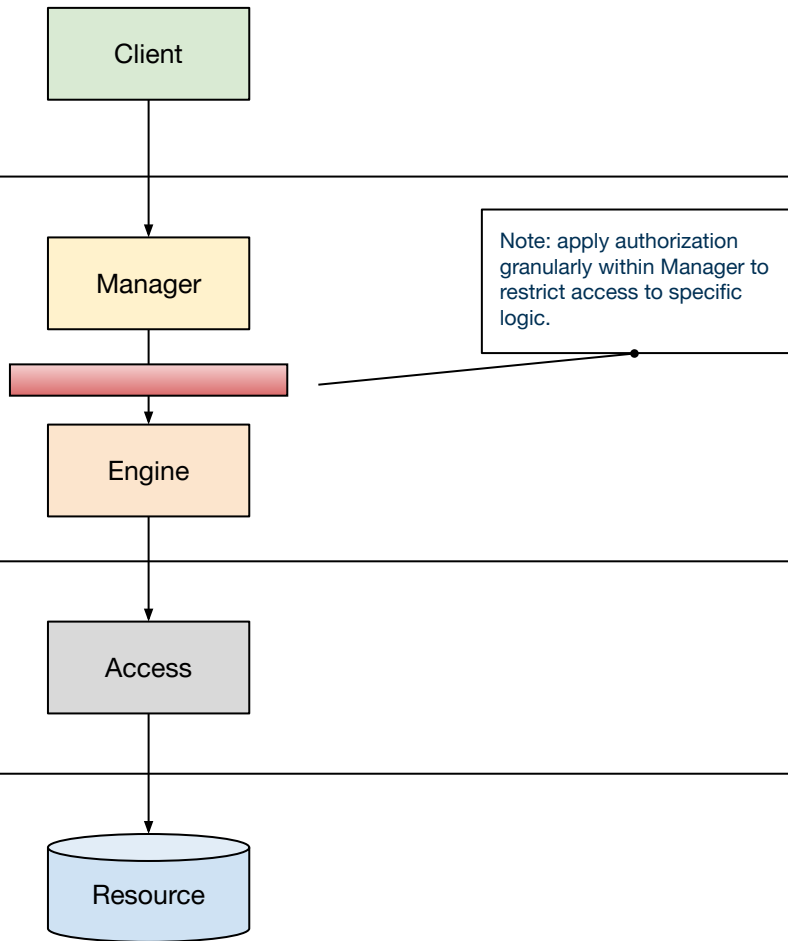


Client

Logic

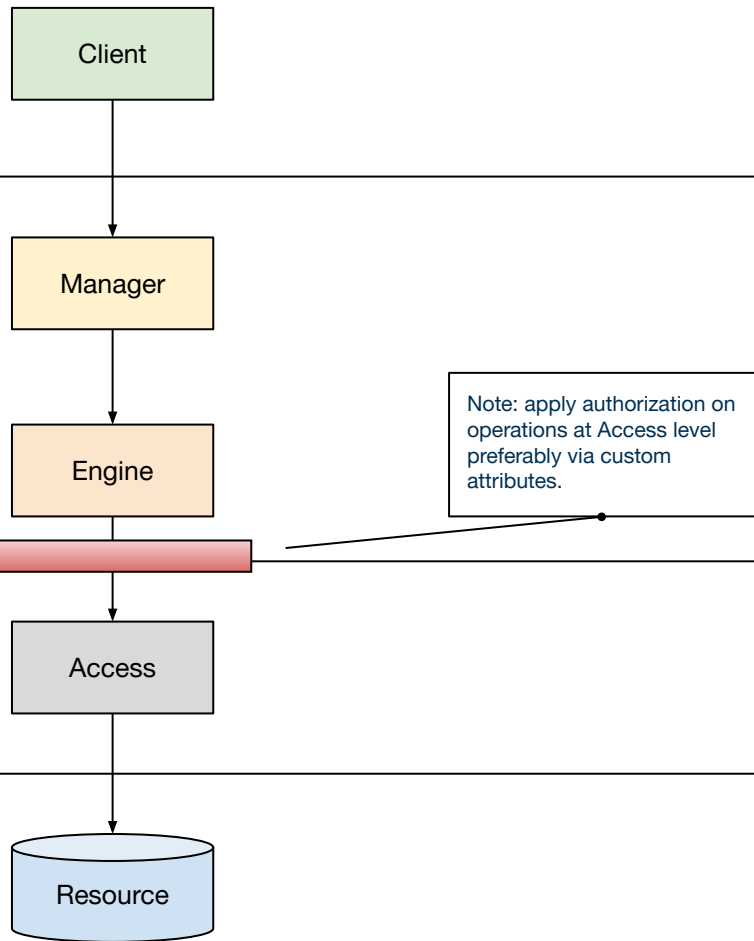
Access

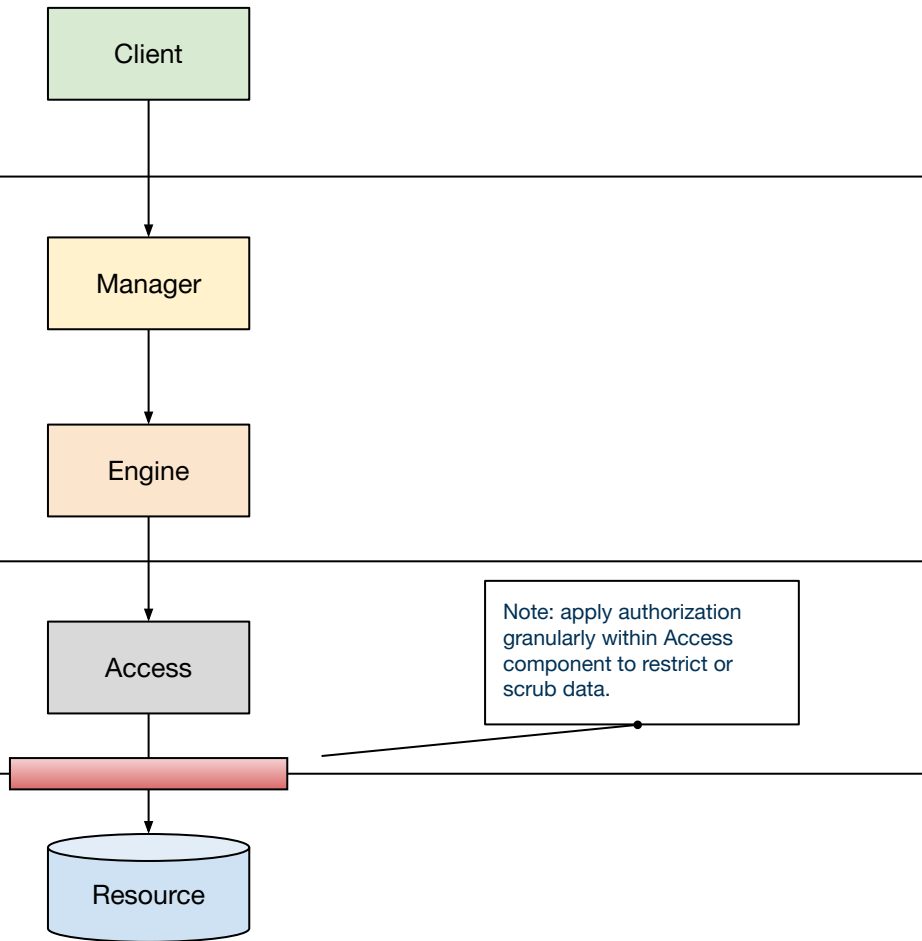
Resource



Authorization - Logic







Project Design

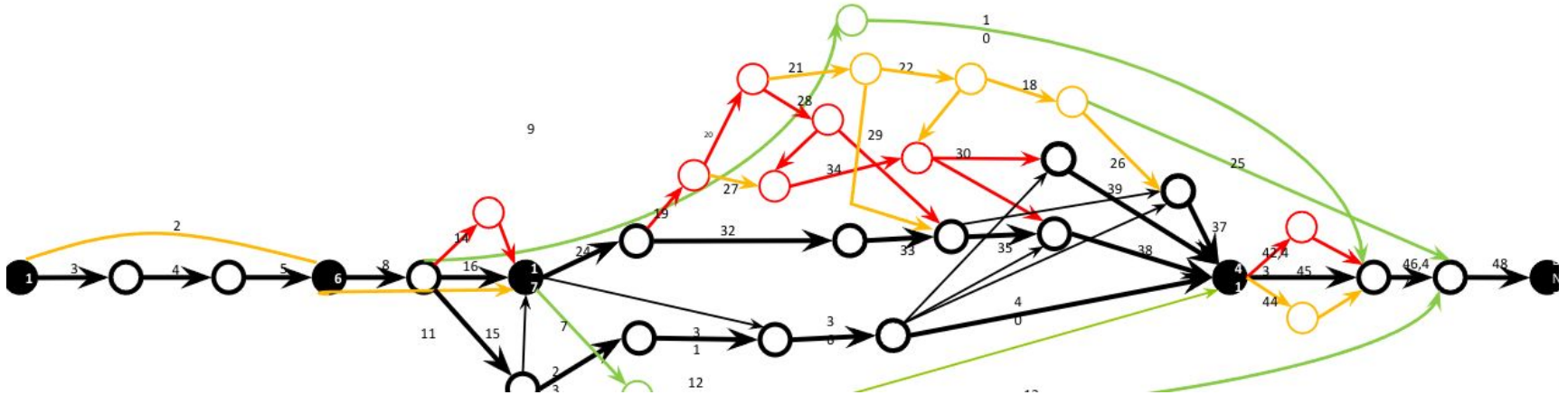


Project Activities

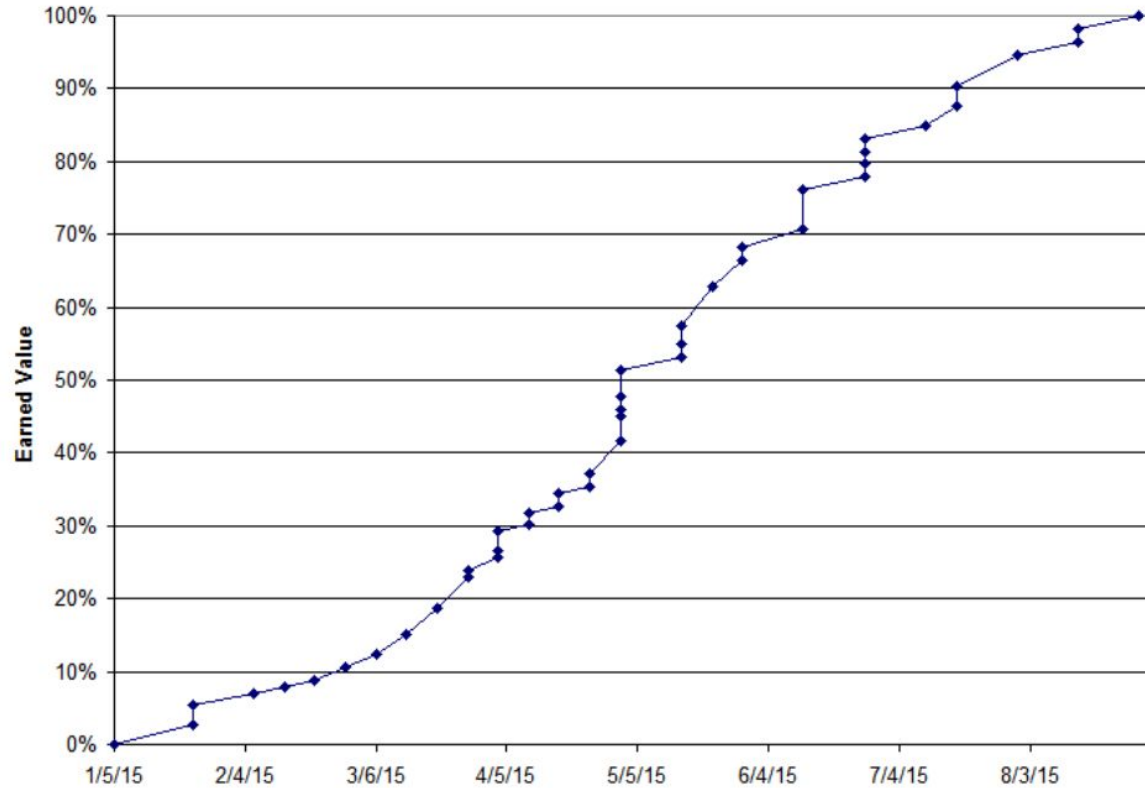
Activities																				
ID	Duration	Name	Dependencies	Resource Dependencies	Dummy	Critical	EST	LST	EFT	LFT	TS	FS	IS	Minimum FS	Minimum EST	Maximum LFT	Target Resources	Ignore Cost	Resource Operator	Allocated to Resour
1	15	Requirements			☐	☒	2/12/2024	2/12/2024	3/4/2024	3/4/2024	0	0	0				Architect,Product Own	☐ No	AND	Architect,Product Own
2	20	Architecture	1	1	☐	☒	3/4/2024	3/4/2024	4/1/2024	4/1/2024	0	0	0				Architect	☐ No	AND	Architect
3	20	Project Planning	2	2	☐	☒	4/1/2024	4/1/2024	4/29/2024	4/29/2024	0	0	0				Architect,Project Man	☐ No	AND	Architect,Project Man
4	30	Test Plan	3		☐	☐	4/29/2024	7/1/2024	6/10/2024	8/12/2024	45	0	45				Test Engineer	☐ No	AND	Test Engineer
5	35	Test Harness	4	4	☐	☐	6/10/2024	8/12/2024	7/29/2024	9/30/2024	45	45	0				Test Engineer	☐ No	AND	Test Engineer
6	15	Logging	3		☐	☒	4/29/2024	4/29/2024	5/20/2024	5/20/2024	0	0	0					☐ No	AND	Dev 1
7	20	Security	3		☐	☐	4/29/2024	5/6/2024	5/27/2024	6/3/2024	5	0	5					☐ No	AND	Dev 2
8	5	Pub/Sub	3		☐	☐	4/29/2024	5/27/2024	5/6/2024	6/3/2024	20	10	10					☐ No	AND	Dev 3
9	20	Resource A	3		☐	☐	4/29/2024	5/6/2024	5/27/2024	6/3/2024	5	0	5				DBA	☐ No	AND	DBA
10	15	Resource B	3		☐	☐	4/29/2024	5/13/2024	5/20/2024	6/3/2024	10	0	10				DBA 2	☐ No	AND	DBA 2
11	10	Access A	6,9	7	☐	☐	5/27/2024	6/3/2024	6/10/2024	6/17/2024	5	0	5					☐ No	AND	Dev 2
12	5	Access B	6,10	8	☐	☐	5/20/2024	6/3/2024	5/27/2024	6/10/2024	10	0	10					☐ No	AND	Dev 3
13	15	Access C	6	6	☐	☒	5/20/2024	5/20/2024	6/10/2024	6/10/2024	0	0	0					☐ No	AND	Dev 1
14	20	Engine A	12,13	11	☐	☐	6/10/2024	6/17/2024	7/8/2024	7/15/2024	5	5	0					☐ No	AND	Dev 2
15	25	Engine B	12,13	13	☐	☒	6/10/2024	6/10/2024	7/15/2024	7/15/2024	0	0	0					☐ No	AND	Dev 1
16	15	Engine C	6	12	☐	☐	5/27/2024	7/1/2024	6/17/2024	7/22/2024	25	20	5					☐ No	AND	Dev 3
17	20	Manager A	7,8,11,14,15	15	☐	☒	7/15/2024	7/15/2024	8/12/2024	8/12/2024	0	0	0					☐ No	AND	Dev 1
18	25	Manager B	7,8,15,16	14	☐	☐	7/15/2024	7/22/2024	8/19/2024	8/26/2024	5	0	5					☐ No	AND	Dev 2
19	25	Client App 1	17,18	18	☐	☐	8/19/2024	8/26/2024	9/23/2024	9/30/2024	5	5	0					☐ No	AND	Dev 2
20	35	Client App 2	17	17	☐	☒	8/12/2024	8/12/2024	9/30/2024	9/30/2024	0	0	0					☐ No	AND	Dev 1
21	30	System Testing	5,19,20		☐	☒	9/30/2024	9/30/2024	11/11/2024	11/11/2024	0	0	0				QC1,QC2	☐ No	AND	QC1,QC2



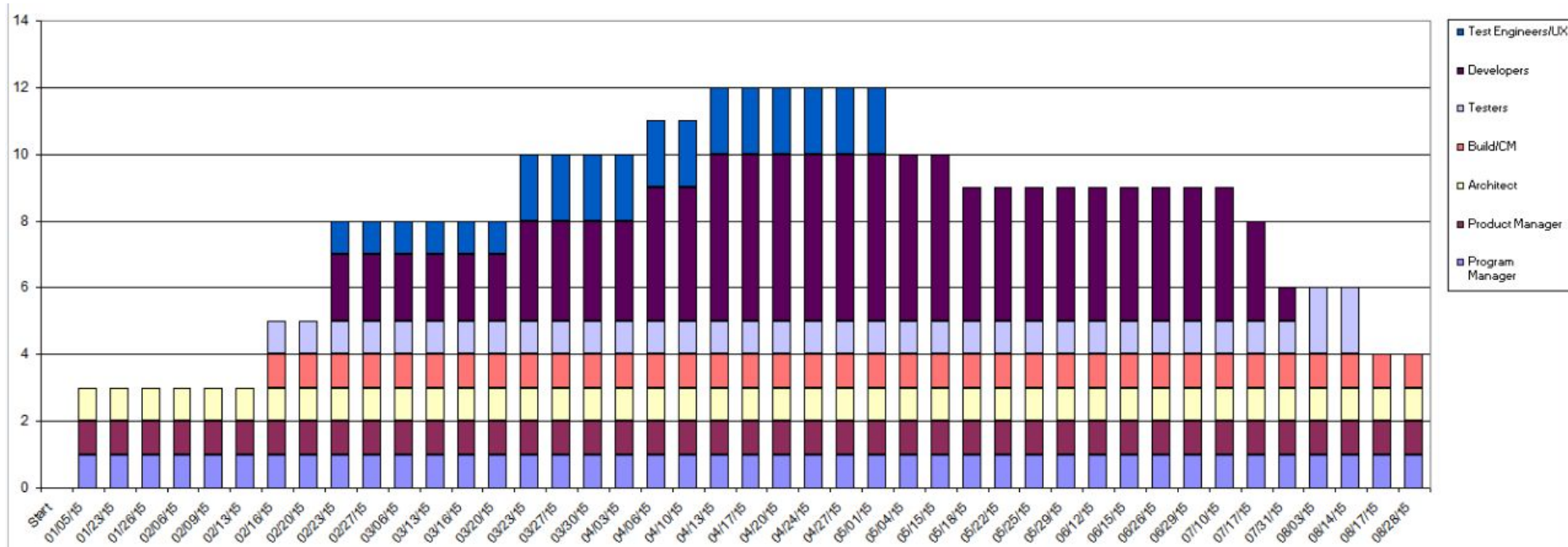
Network Analysis



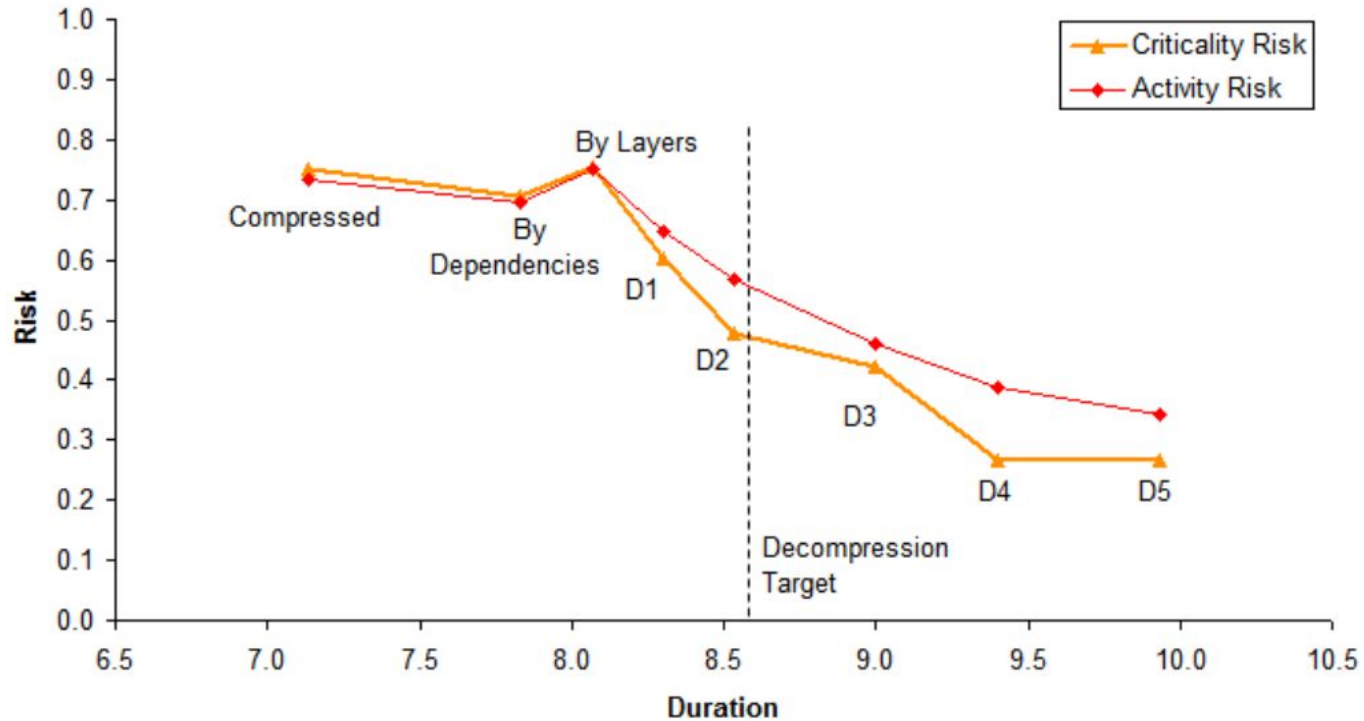
Planned Progress



Staffing Distribution



Risk Index



Project Design Options

Description	Duration	Cost	Risk	Complexity
Top Developer	8	61	0.74	High
Decompression	9	68	0.38	Low
Understaffed	12	80	0.47	Low



Planning Assumptions

Require full time

- 1 Architect
- 1 Product Manager
- 1 Project Manager
- X Developers
- 1 Test Engineer
- 2 Testers
- 1 Dev Ops
- 1 UX



Planning Assumptions

Access to Specialists (if any)

- Junior Architect
- Security
- DBA
- UX / UI Design
- Mobile development
- Etc.



Risks



Architecture

Anti-Design



Component Taxonomy (Anti-design)

Caller

- Originator of a request
- An app, API or another system
- In most software systems, complex orchestration logic ends up here

Thing

- A thing (“microservice”?) that does stuff
- This is not a Manager or Engine because it has not be decomposed by volatility, it does not consider information hiding and dependency management, and it does not honor cost vs. count

Goo

- Goo between clients, things and stuff
- Not truly utilities or infrastructure, but an attempt to provide them

Stuff

- Stuff you stick somewhere and retrieve when you want it
- These are resources and in an anti-design, it is quite possible that a Caller goes straight to Stuff, Stuff will also go straight to other Stuff, and some Stuff may even interact directly with Things!

Stuff API

File Stuff



Caller

Thing

Goo

Stuff

Stuff API

File Stuff

